

Feature Detection and Matching

Computer Vision: AI3619

Feature Detection and Matching

Finding interest points in real-world cluttered scenes and matching them

Topics:

- (1) Motivating Feature Detection
- (2) Local Appearance and Interest Points
- (3) Blob Detection
- (4) Scale-Space
- (5) The SIFT interest point detector
- (6) Matching and Results
- (7) Histogram-of-Oriented Gradients (HoG) / Shape Contexts

Motivation: Build a Panorama



Motivation: Build a Panorama

Motivation: Build a Panorama



M. Brown and D. G. Lowe. Recognising Panoramas. ICCV 2003

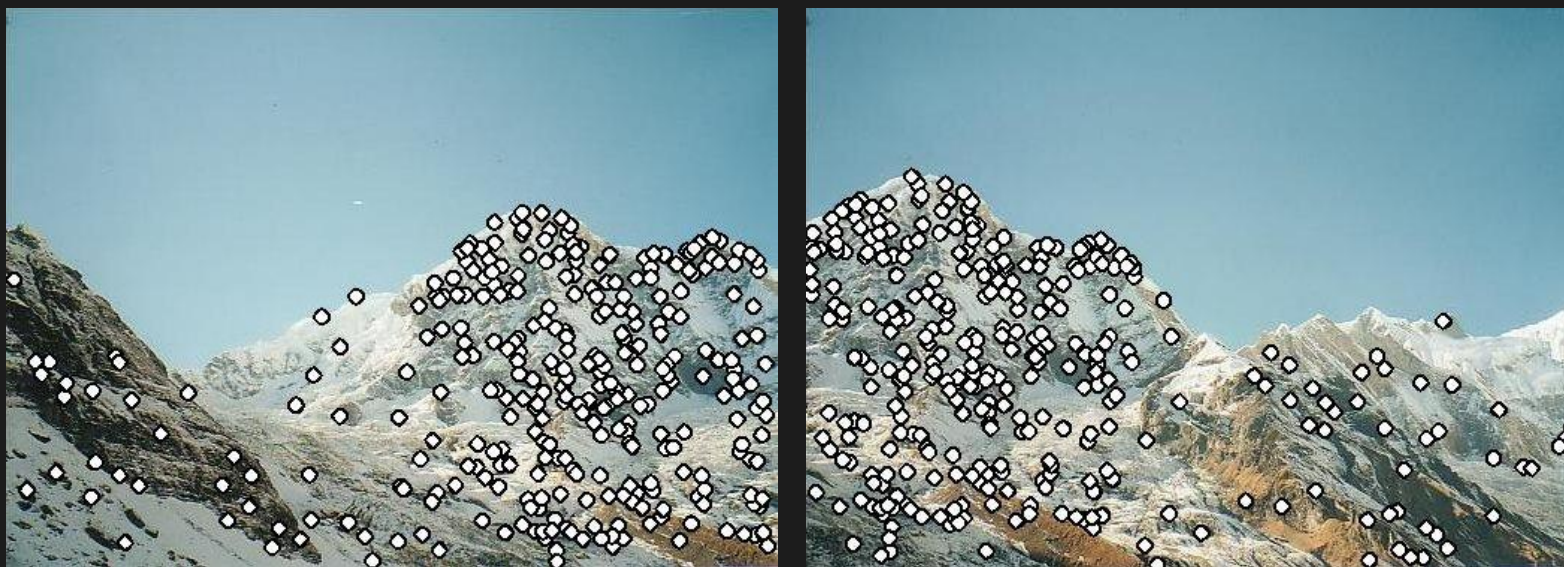
How do we build a panorama?

- We need to match (align) images



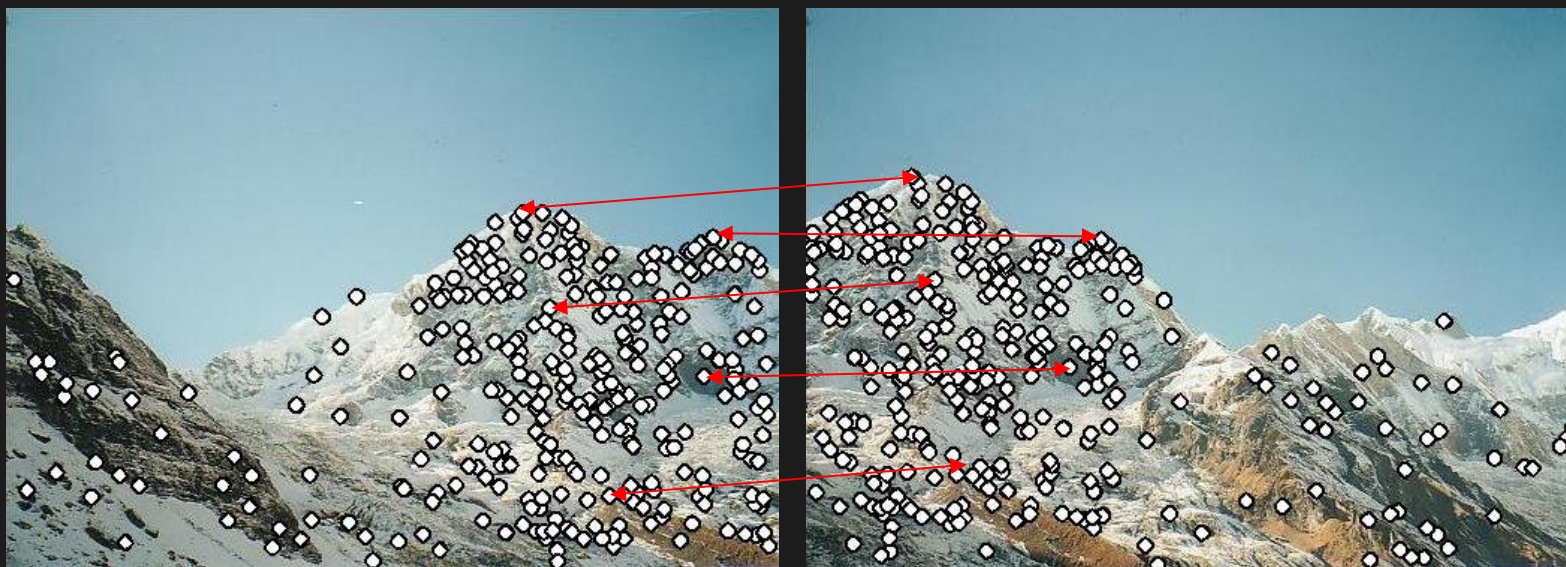
Matching with Features

- Detect feature points in both images



Matching with Features

- Detect feature points in both images
- Find corresponding pairs (Matching)



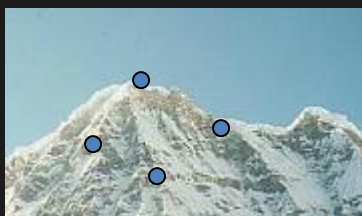
Matching with Features

- Detect feature points in both images
 - Find corresponding pairs
- Use these pairs to align images



Matching with Features

- Problem 1:
 - Detect the *same* point *independently* in both images



no chance to match!

We need a repeatable detector

Matching with Features

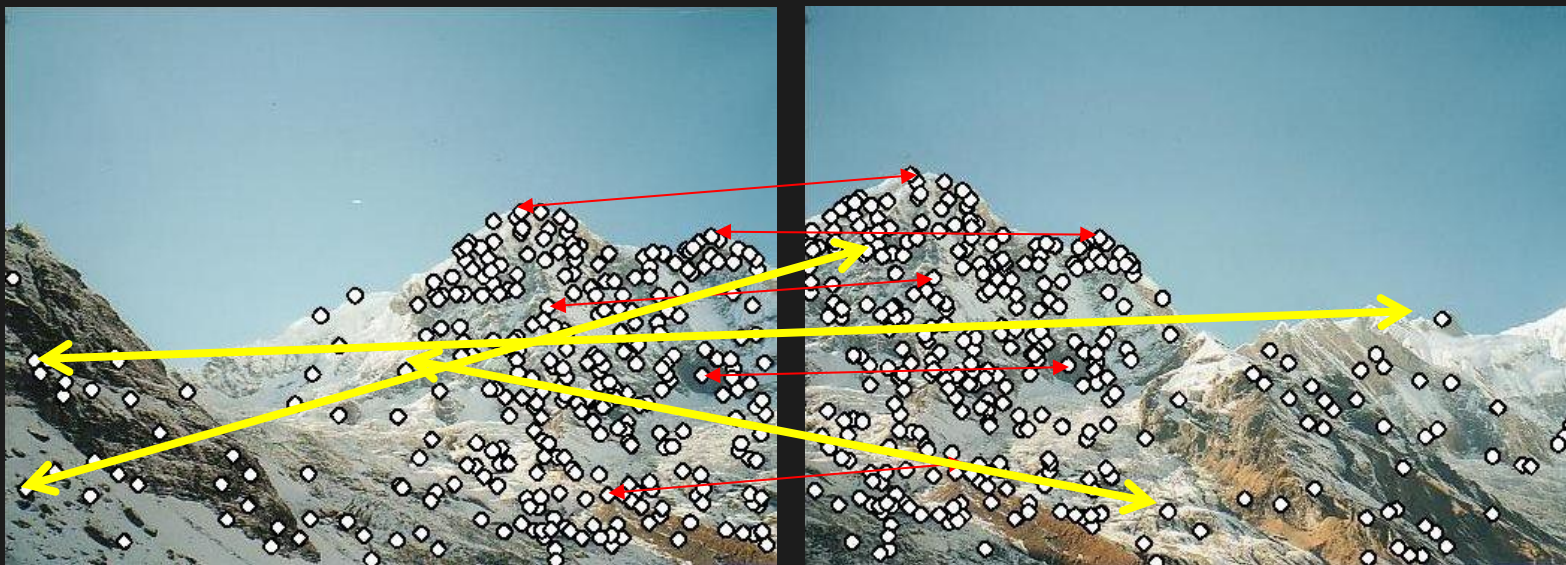
- Problem 2:
 - For each point correctly recognize the corresponding one



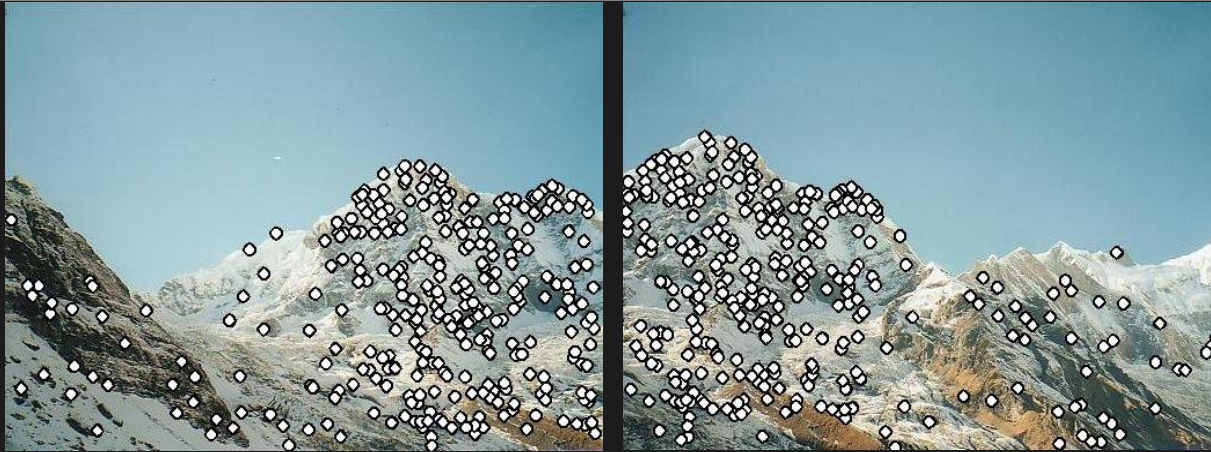
We need a reliable and distinctive descriptor

Matching with Features

- Problem 3:
 - Need to estimate transformation between images, despite erroneous correspondences.



Characteristics of Good Features



- **Repeatability**
 - The same feature can be found in several images despite geometric and photometric transformations
- **Saliency**
 - Each feature has a distinctive description
- **Compactness and efficiency**
 - Many fewer features than image pixels
- **Locality**
 - A feature occupies a relatively small area of the image; robust to clutter and occlusion

Applications

- Feature points are used for:
 - Motion tracking
 - Image alignment
 - 3D reconstruction
 - Object recognition
 - Indexing and database retrieval
 - Robot navigation

A Little Quiz

How to find and match “Interesting Points or Features”?



Template



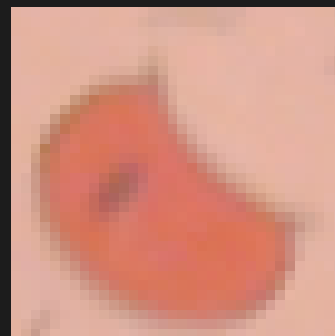
Rich 2D Image

Raw Images are Hard to Match



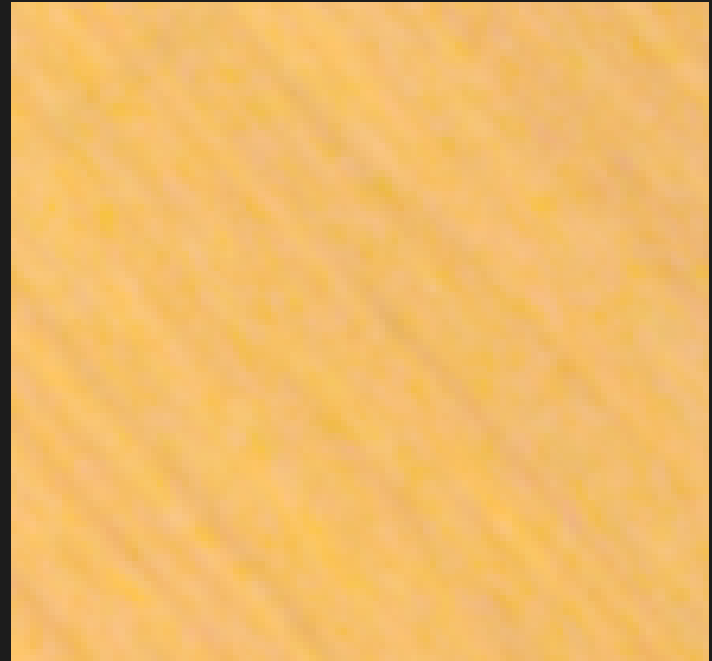
Different Size, Orientation, Lighting, Brightness, ...

Removing Sources of Variation



Matching becomes easier if we can
remove variations like size and orientation.

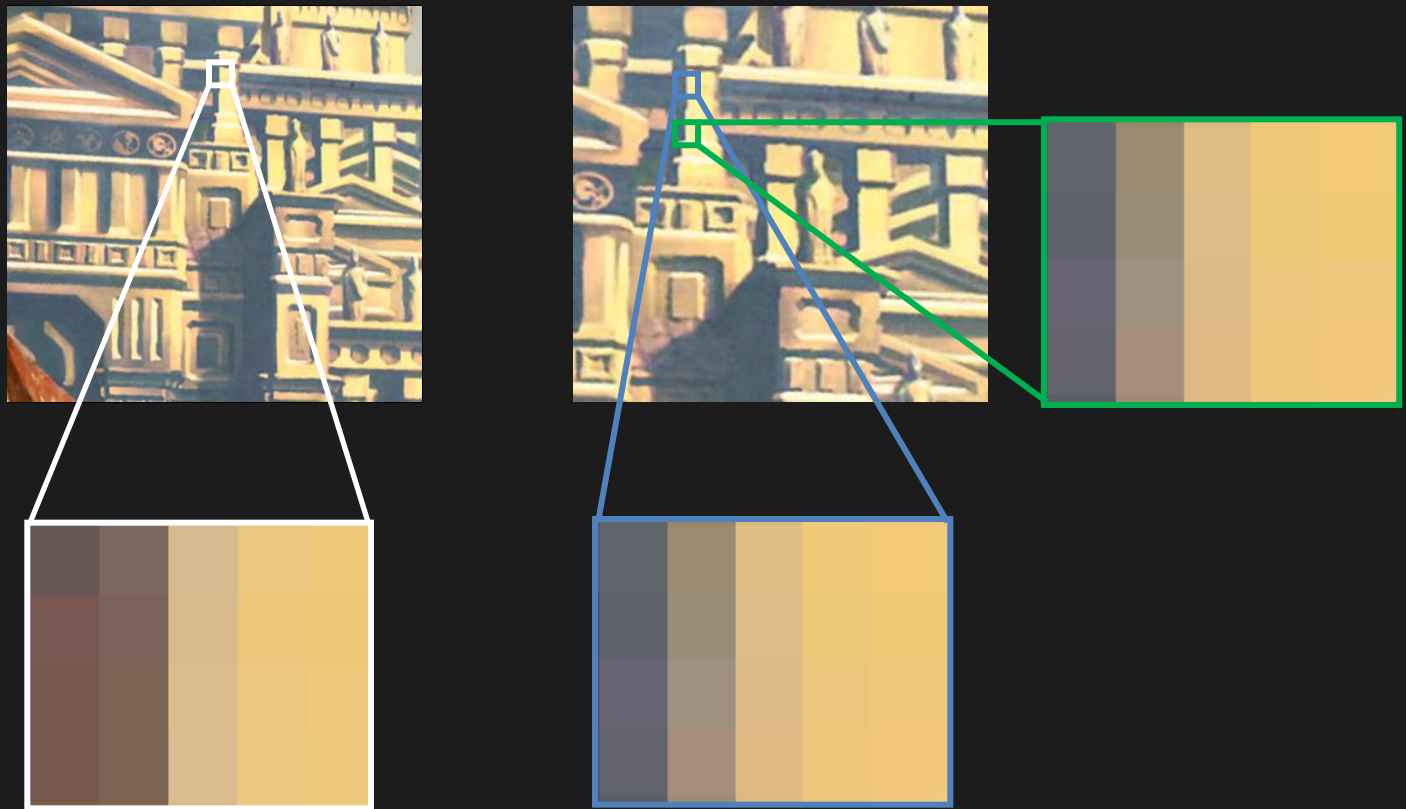
Some Patches are not “Interesting”



What is an Interesting Point/Feature?

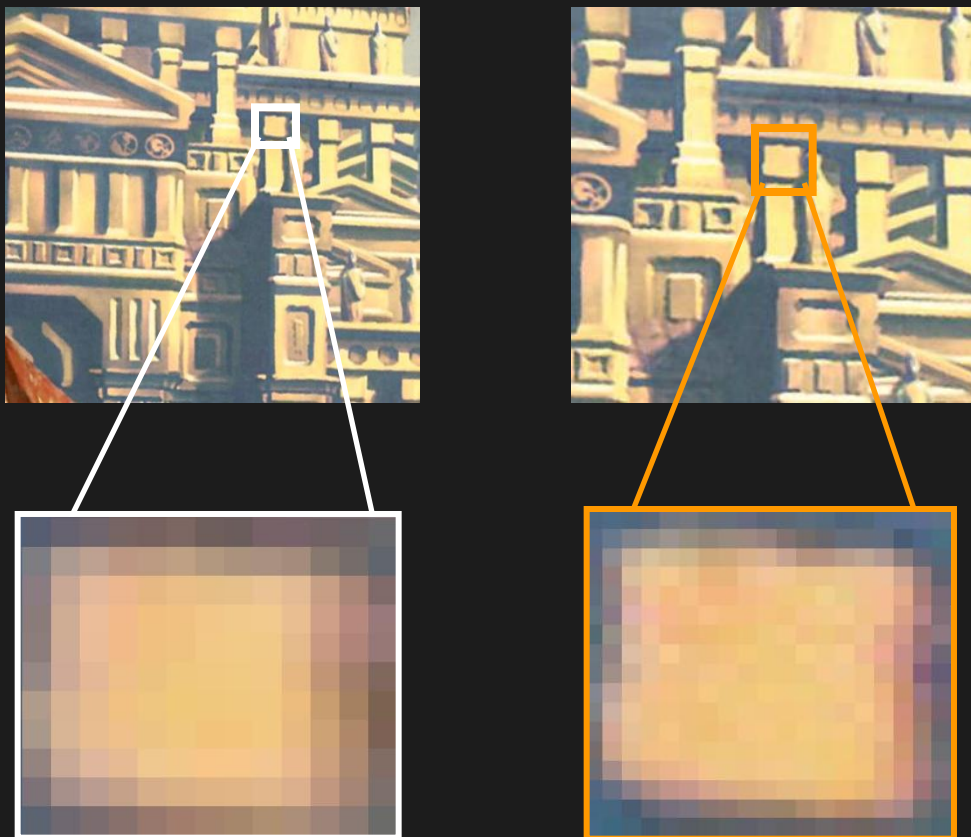
- Has **rich image content** (color variations, gradient variations, ...) within the local window
- Has well defined **representation (signature)** for matching/Comparing with other points
- Has a well defined **position** in the image
- Should be **invariable to image rotation and scaling**
- Should be relatively **invariable to lighting** changes

Are Lines/Edges Interesting?



Cannot “**Localize**” an Edge

Are Blobs Interesting?



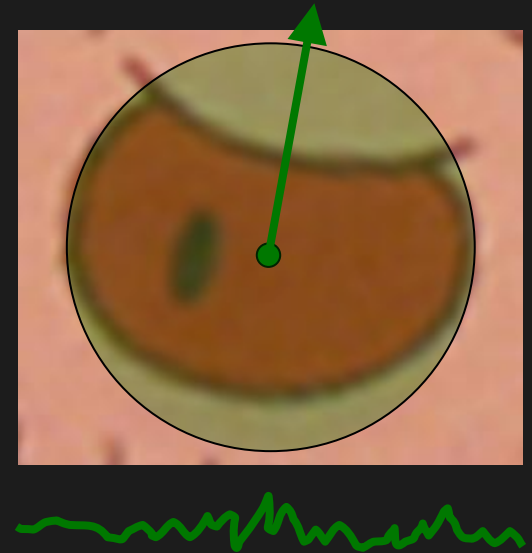
Yes! Blobs have **fixed position** and definite **size**.

Blobs as Interest Points

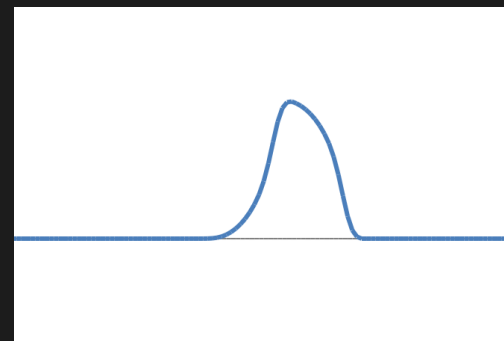
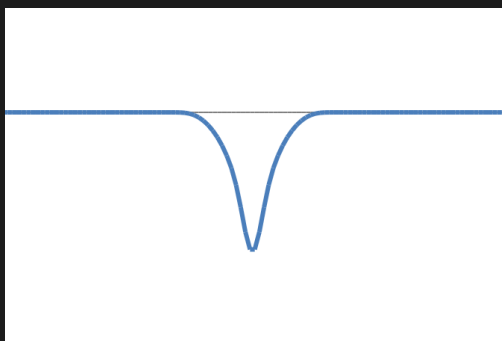
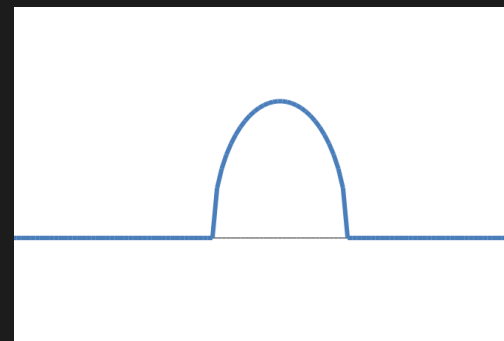
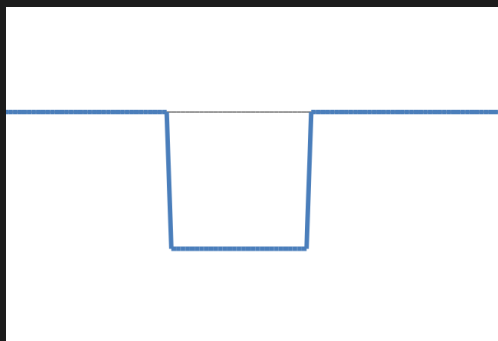
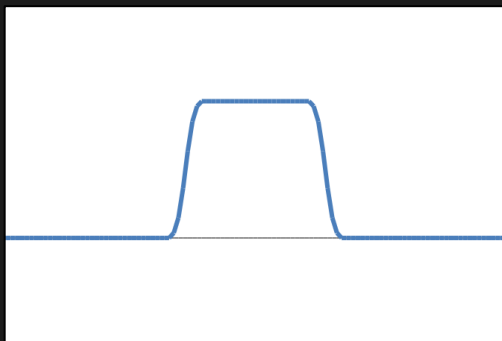
We will use **Blob-like Features** for 2D recognition.

We need to:

- **Locate** a blob
- Determine its **size**
- Determine its **orientation**
- Formulate a **description** or signature that is independent of size and orientation

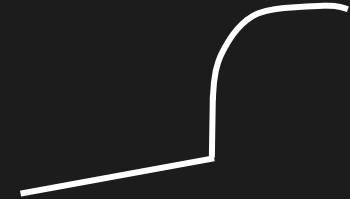
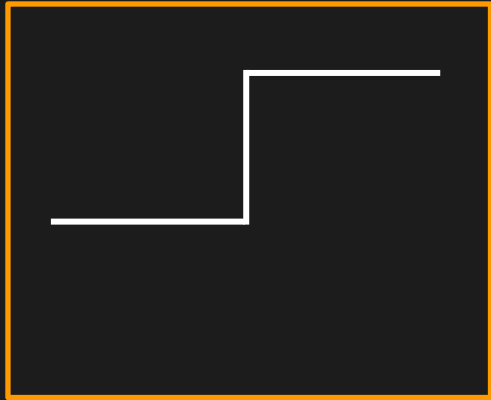


1D Blobs



Examples of 1D Blob-like structures

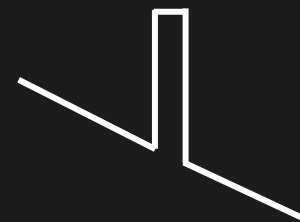
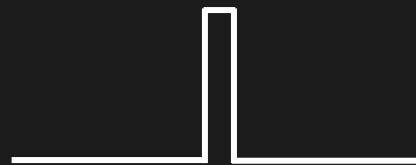
Types of Edges



Step Edges



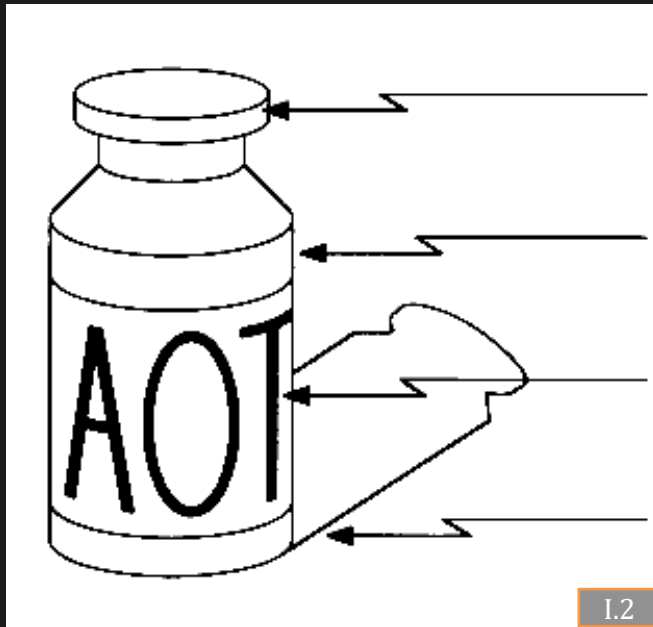
Roof Edge



Line Edges

Causes of Edges

Edges are caused by a variety of factors



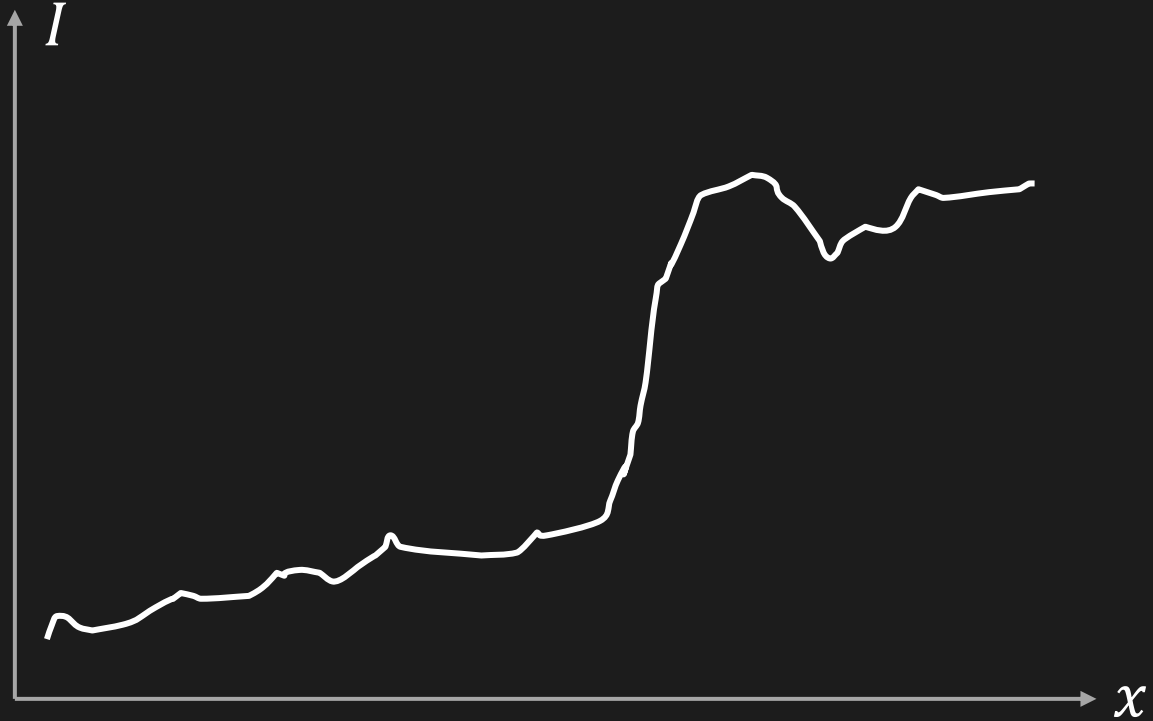
Surface Normal Discontinuity

Depth Discontinuity

Surface Color Discontinuity

Illumination Discontinuity

Real Edges



Problems: **Noisy** Images and **Discrete** Images

Edge Detector

We want an **Edge Operator** that produces:

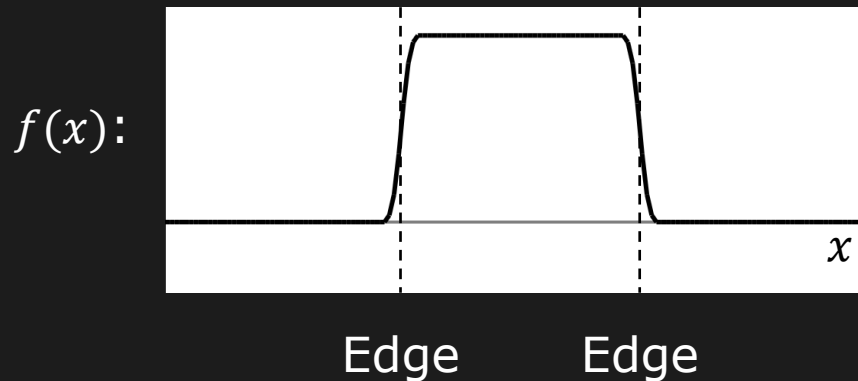
- Edge **Position**
- Edge **Magnitude** (Strength)
- Edge **Orientation** (Direction)

Crucial Requirements:

- High **Detection Rate**
- Good **Localization**
- **Robust** to Noise

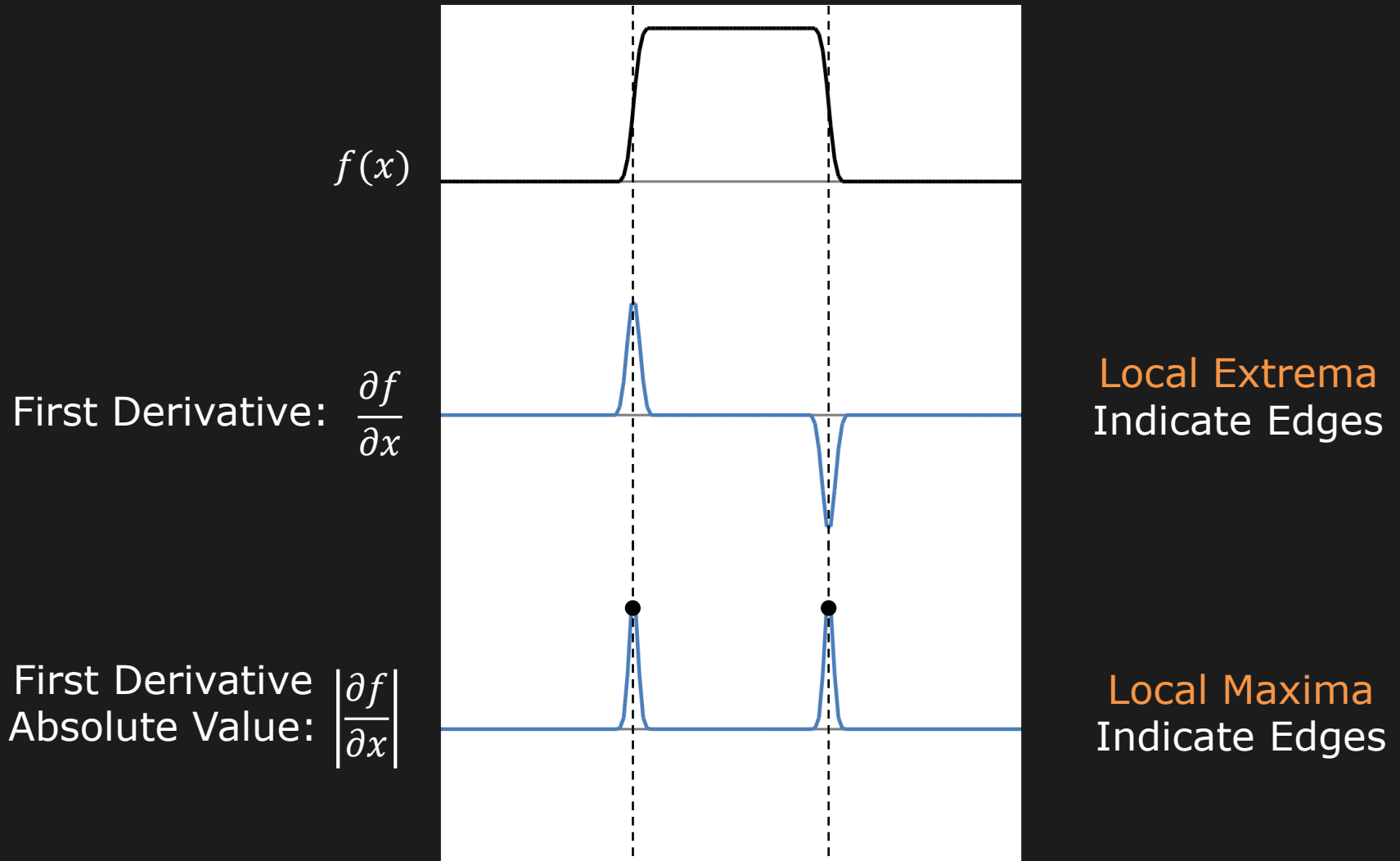
1D Edge Detection

Edges are rapid changes in image brightness in a small region.



Calculus 101: Derivative of a continuous function represents the amount of change in the function.

Edge Detection Using 1st Derivative



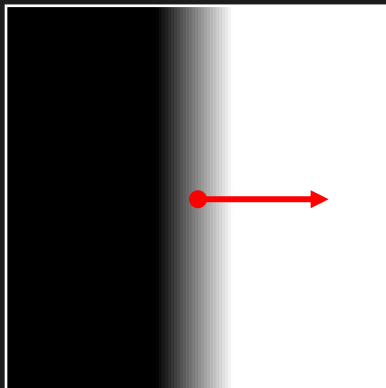
Provides Both Location and Strength of an Edge

Gradient (∇)

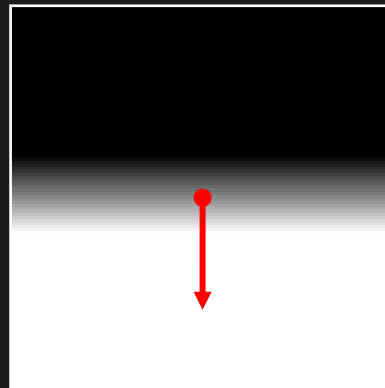
Gradient (Partial Derivative) Represents the Direction of Most Rapid Change in Intensity

$$\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$$

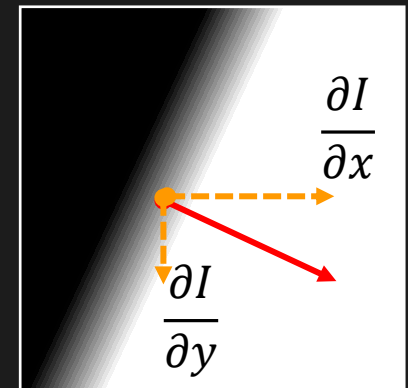
Pronounced as “Del I”



$$\nabla I = \left[\frac{\partial I}{\partial x}, 0 \right]$$



$$\nabla I = \left[0, \frac{\partial I}{\partial y} \right]$$

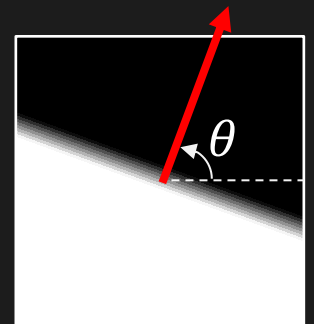


$$\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$$

Gradient (∇) as Edge Detector

Gradient Magnitude $S = \|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$

Gradient Orientation $\theta = \tan^{-1} \left(\frac{\partial I}{\partial y} / \frac{\partial I}{\partial x} \right)$

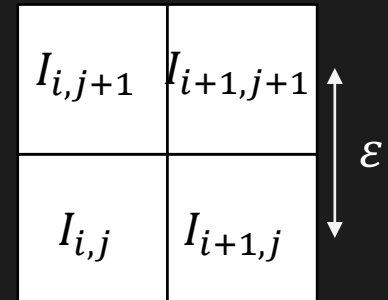


Discrete Gradient (∇) Operator

Finite difference approximations:

$$\frac{\partial I}{\partial x} \approx \frac{1}{2\varepsilon} \left((I_{i+1,j+1} - I_{i,j+1}) + (I_{i+1,j} - I_{i,j}) \right)$$

$$\frac{\partial I}{\partial y} \approx \frac{1}{2\varepsilon} \left((I_{i+1,j+1} - I_{i+1,j}) + (I_{i,j+1} - I_{i,j}) \right)$$



Can be implemented as Convolution!

$$\frac{\partial}{\partial x} \approx \frac{1}{2\varepsilon} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$\frac{\partial}{\partial y} \approx \frac{1}{2\varepsilon} \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

Note: Convolution flips have been applied

Comparing Gradient (∇) Operators

Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	0 1 -1 0	-1 0 1 -1 0 1 -1 0 1	-1 0 1 -2 0 2 -1 0 1	-1 -2 0 2 1 -2 -3 0 3 2 -3 -5 0 5 3 -2 -3 0 3 2 -1 -2 0 2 1
$\frac{\partial I}{\partial y}$	1 0 0 -1	1 1 1 0 0 0 -1 -1 -1	1 2 1 0 0 0 -1 -2 -1	1 2 3 2 1 2 3 5 3 2 0 0 0 0 0 -2 -3 -5 -3 -2 -1 -2 -3 -2 -1

Good Localization

Noise Sensitive

Poor Detection



Poor Localization

Less Noise Sensitive

Good Detection

Gradient (∇) Using Sobel Filter



Image (I)



$\partial I / \partial x$

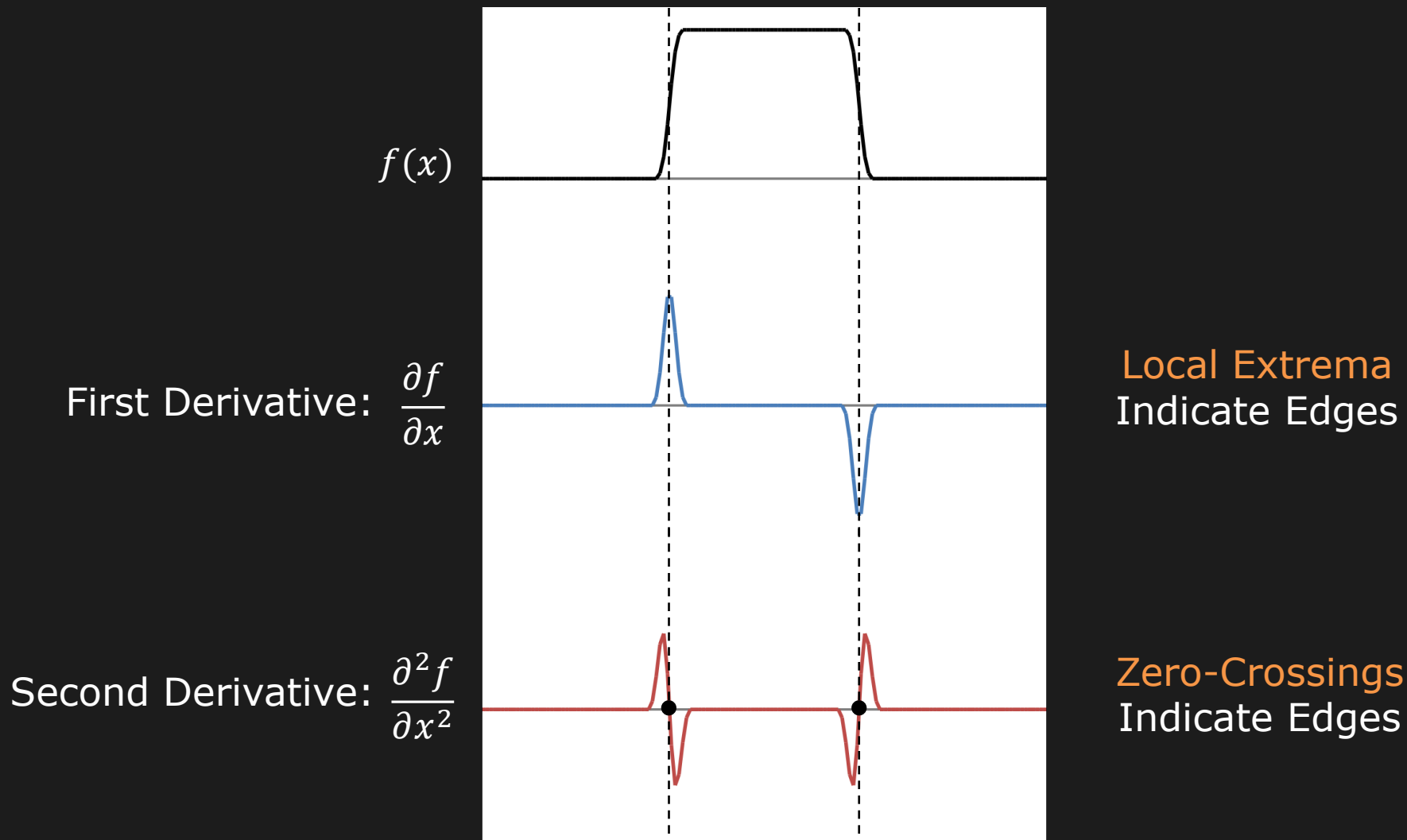


$\partial I / \partial y$



Gradient Magnitude

Edge Detection Using 2nd Derivative



Provides Only the Location of an Edge

Laplacian (∇^2) as Edge Detector

Laplacian: Sum of Pure Second Derivatives

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

Pronounced as “Del Square I ”

“Zero-Crossings” in Laplacian of an image represent edges

Discrete Laplacian(∇^2) Operator

Finite difference approximations:

$$\frac{\partial^2 I}{\partial x^2} \approx \frac{1}{\varepsilon^2} (I_{i-1,j} - 2I_{i,j} + I_{i+1,j})$$

$$\frac{\partial^2 I}{\partial y^2} \approx \frac{1}{\varepsilon^2} (I_{i,j-1} - 2I_{i,j} + I_{i,j+1})$$

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

$I_{i-1,j+1}$	$I_{i,j+1}$	$I_{i+1,j+1}$	ε
$I_{i-1,j}$	$I_{i,j}$	$I_{i+1,j}$	
$I_{i-1,j-1}$	$I_{i,j-1}$	$I_{i+1,j-1}$	

Convolution Mask :

$$\nabla^2 \approx \frac{1}{\varepsilon^2} \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

OR

$$\nabla^2 \approx \frac{1}{6\varepsilon^2} \begin{bmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{bmatrix}$$

(More Accurate)

Laplacian Edge Detector



Image (I)



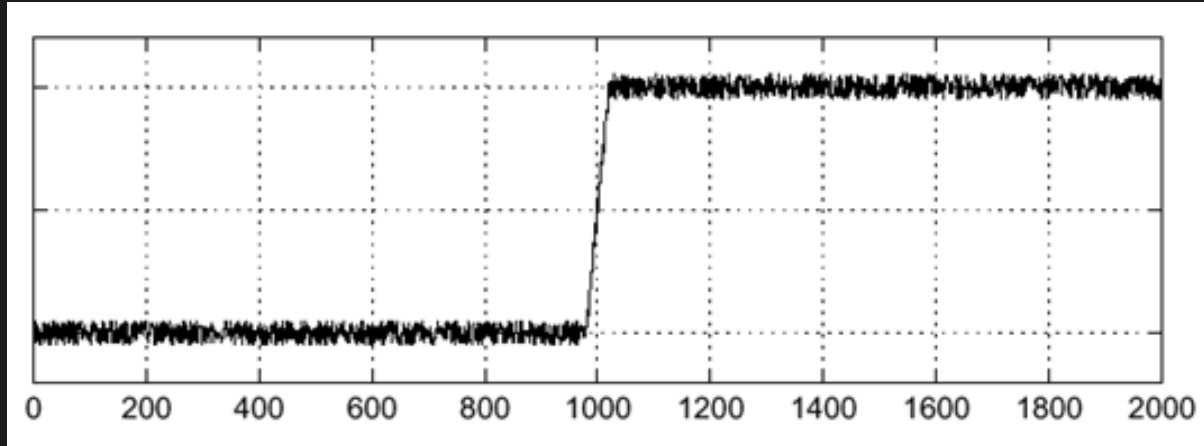
Laplacian
(0 maps to 128)



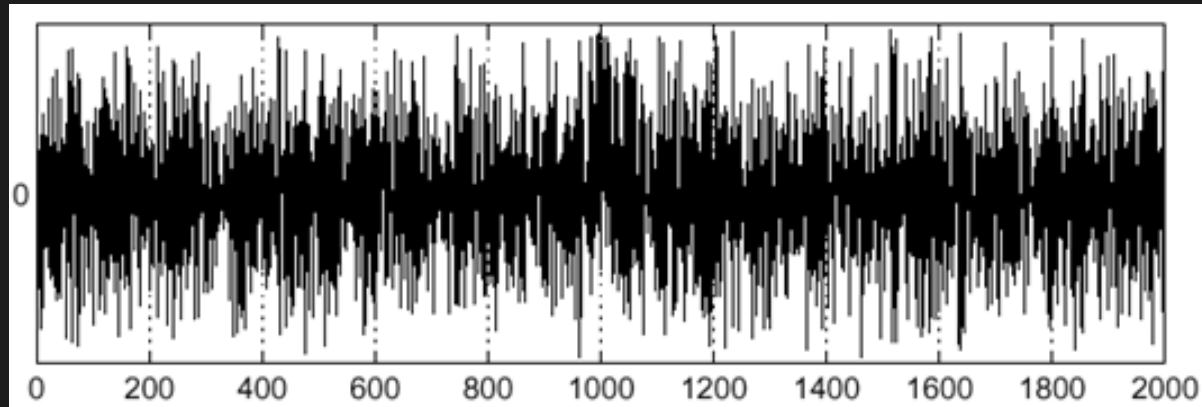
Laplacian
"Zero Crossings"

Effects of Noise

$f(x)$



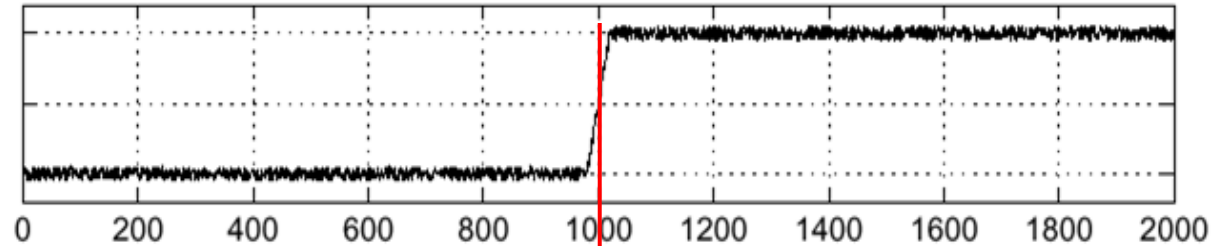
$\nabla f(x)$
(Gradient)



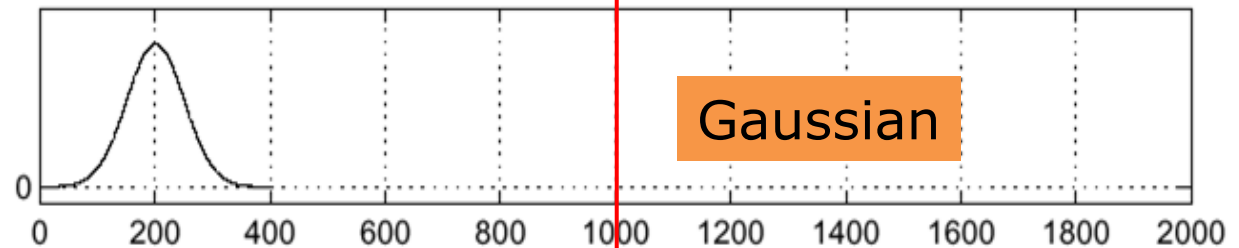
Where is the edge??

Solution: Gaussian Smooth First

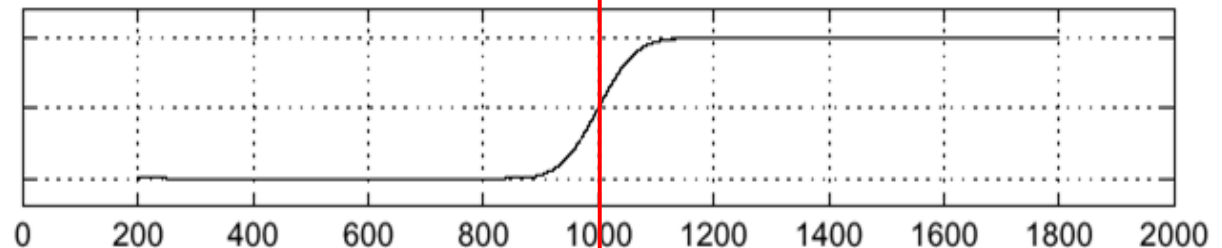
f



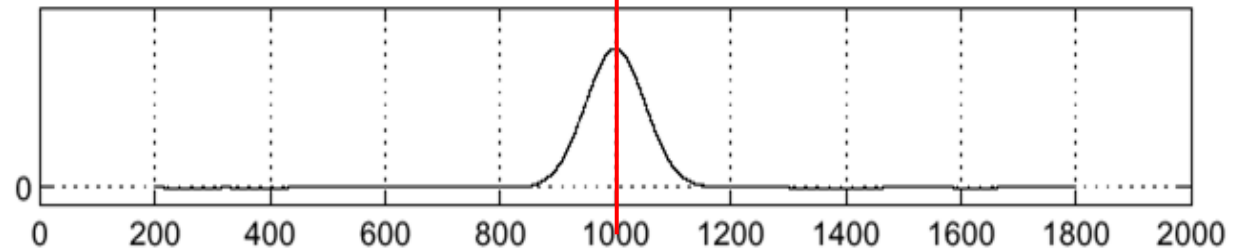
n_σ



$n_\sigma * f$



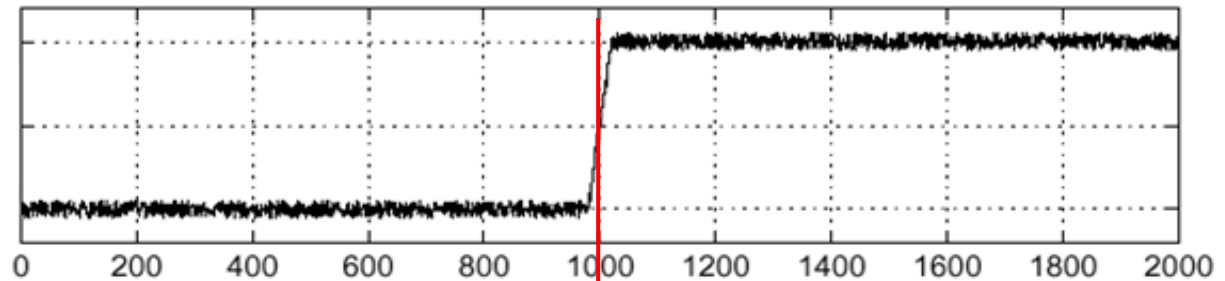
$\nabla(n_\sigma * f)$



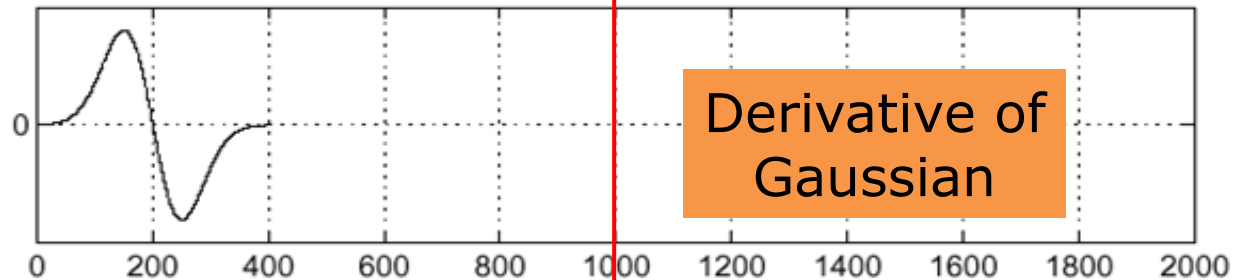
Derivative of Gaussian ($\nabla(n_\sigma)$)

$\nabla(n_\sigma * f) = \nabla(n_\sigma) * f$...saves us one operation.

f

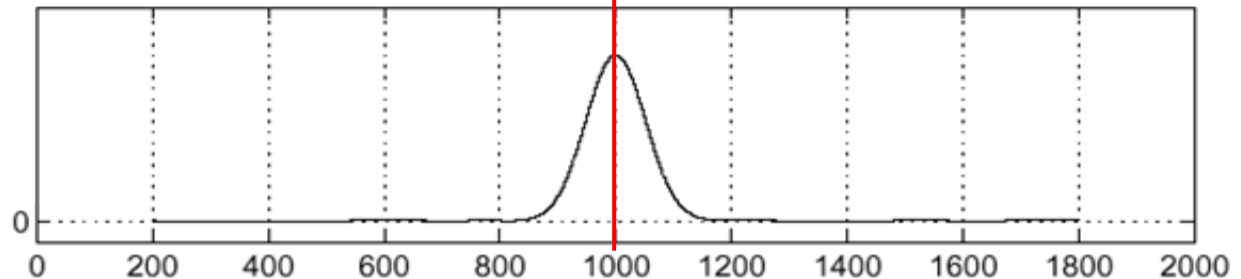


$\nabla(n_\sigma)$



Derivative of
Gaussian

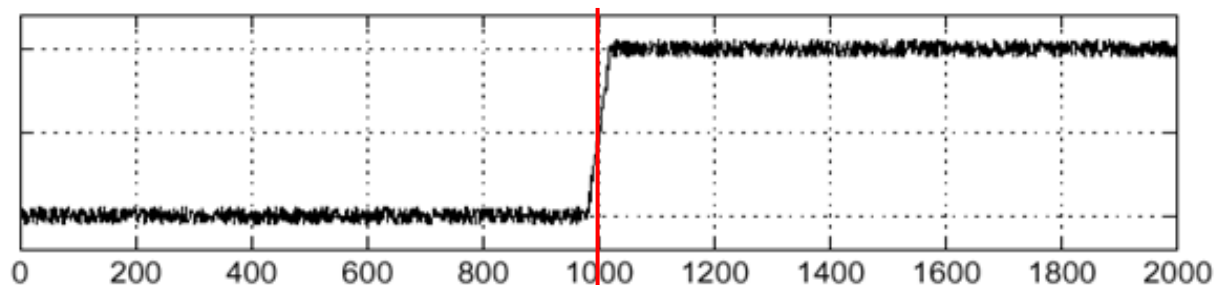
$\nabla(n_\sigma) * f$



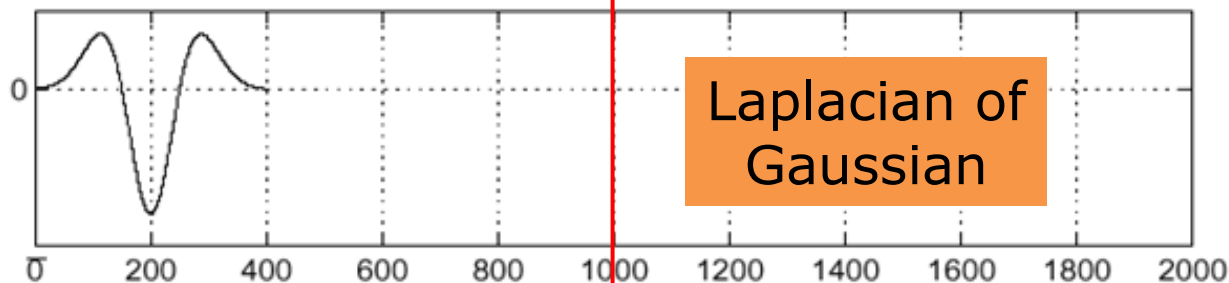
Laplacian of Gaussian ($\nabla^2 n_\sigma$ or $\nabla^2 G$)

$\nabla^2(n_\sigma * f) = \nabla^2(n_\sigma) * f$...saves us one operation.

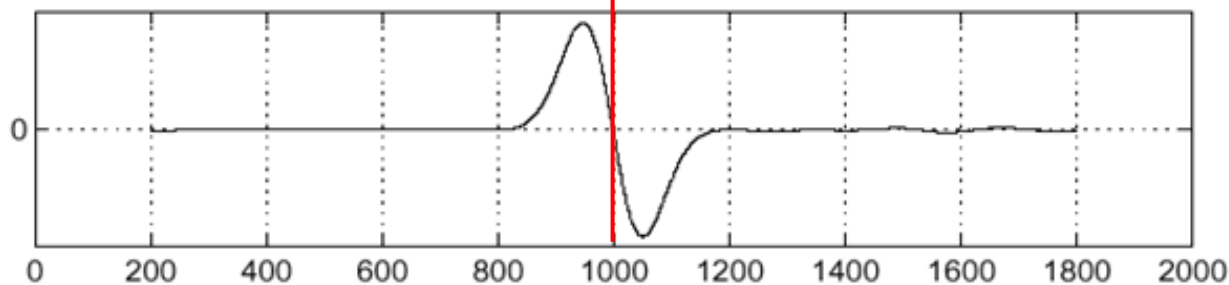
f



$\nabla^2(n_\sigma)$

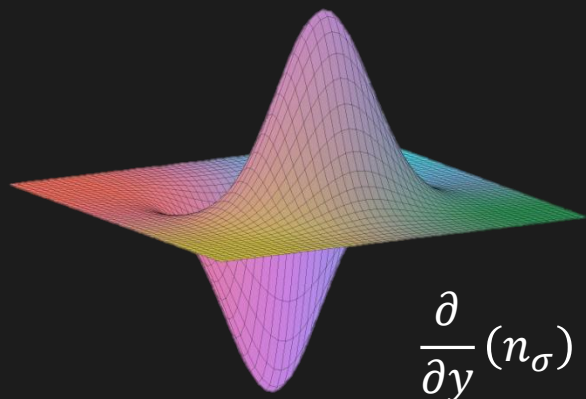
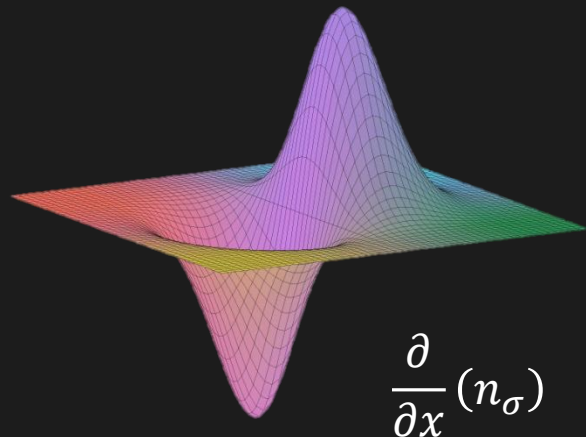


$\nabla^2(n_\sigma) * f$

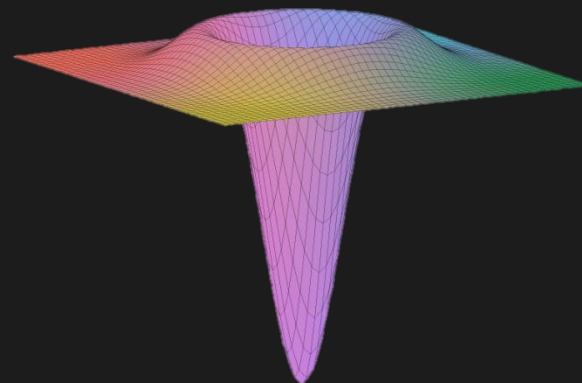


Gradient vs. Laplacian

Derivative of Gaussian (∇G)



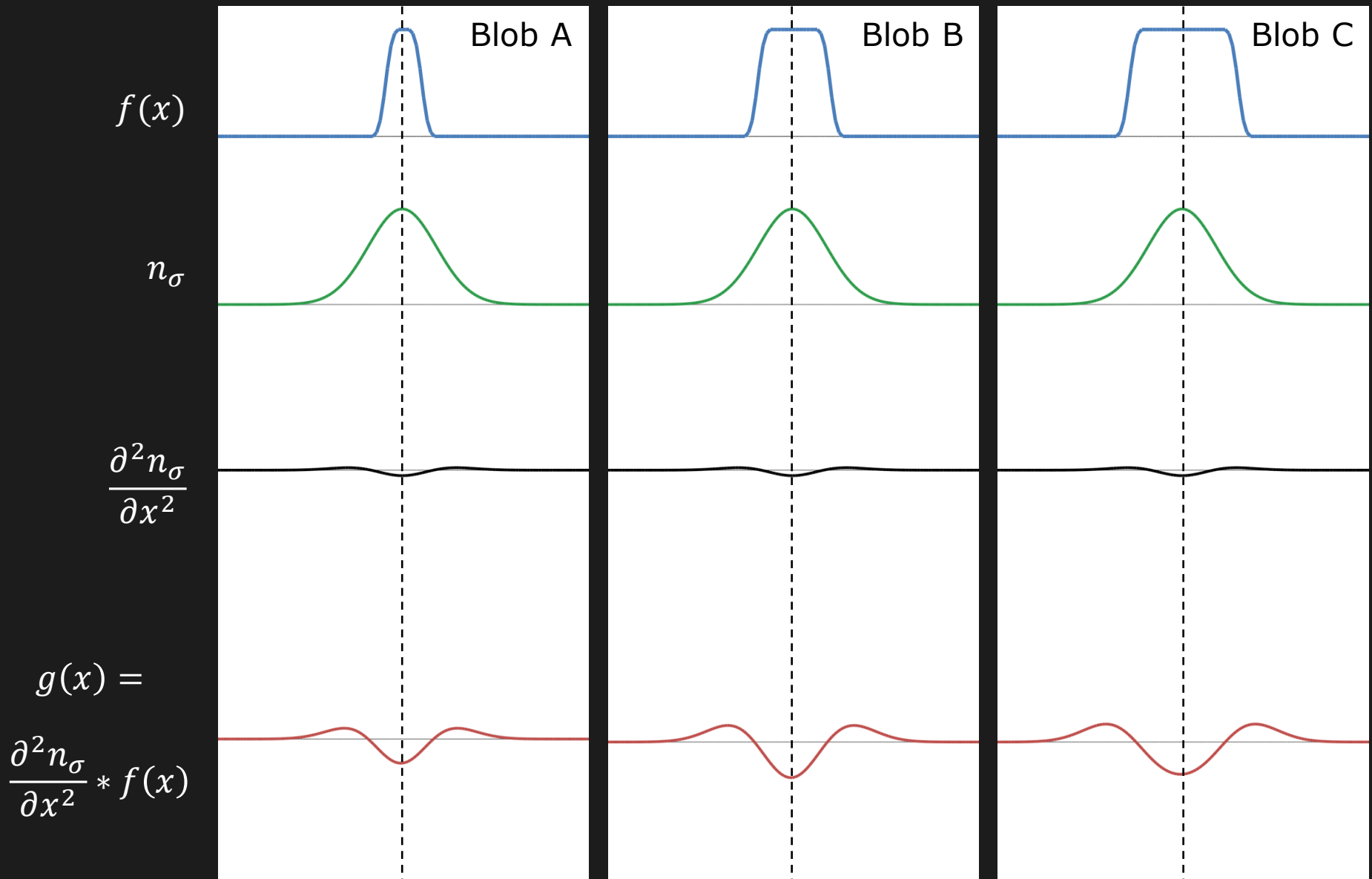
Laplacian of Gaussian ($\nabla^2 G$)



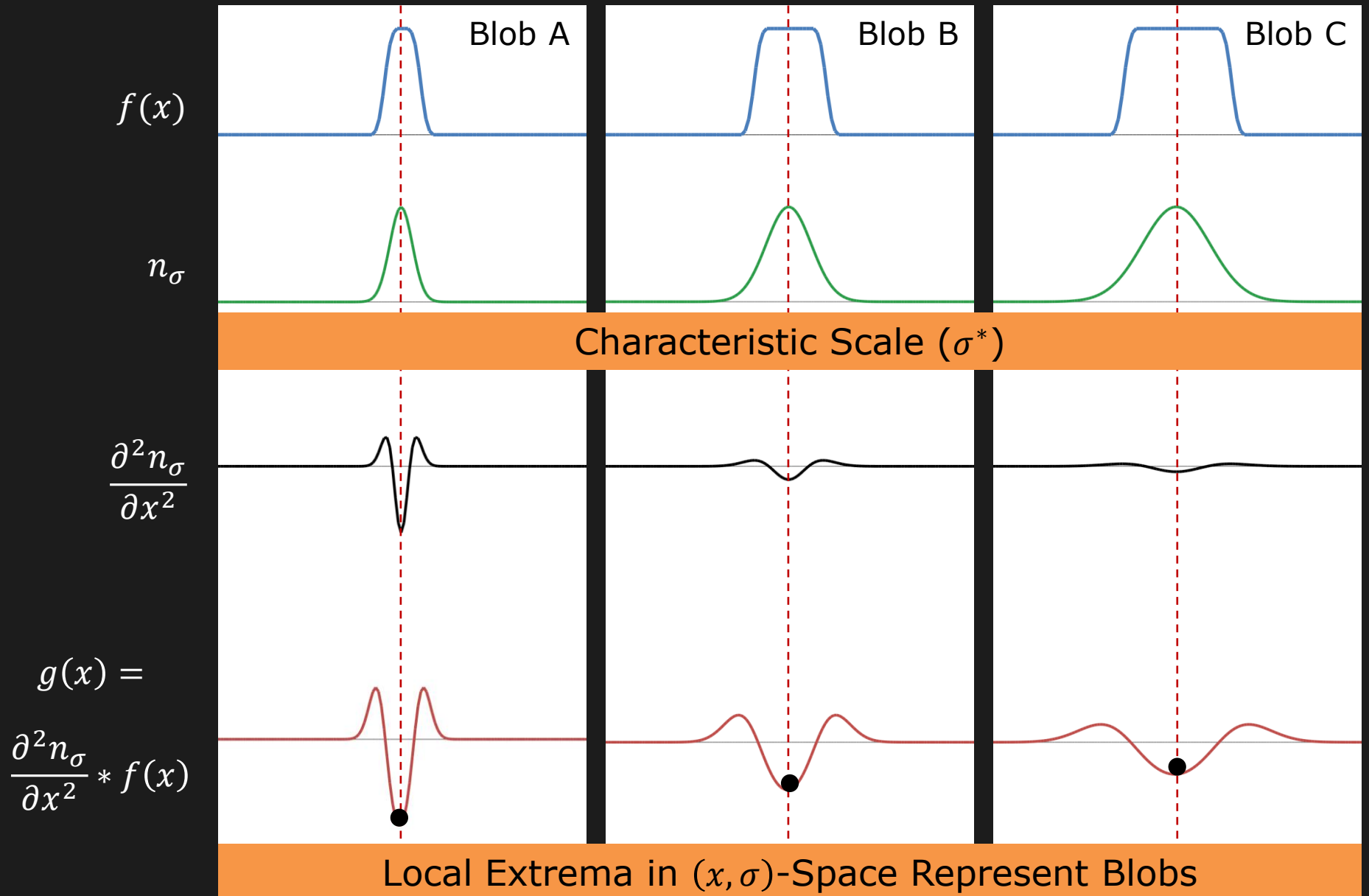
Inverted "Sombrero"
(Mexican Hat)

$$\frac{\partial^2}{\partial x^2}(n_\sigma) + \frac{\partial^2}{\partial y^2}(n_\sigma)$$

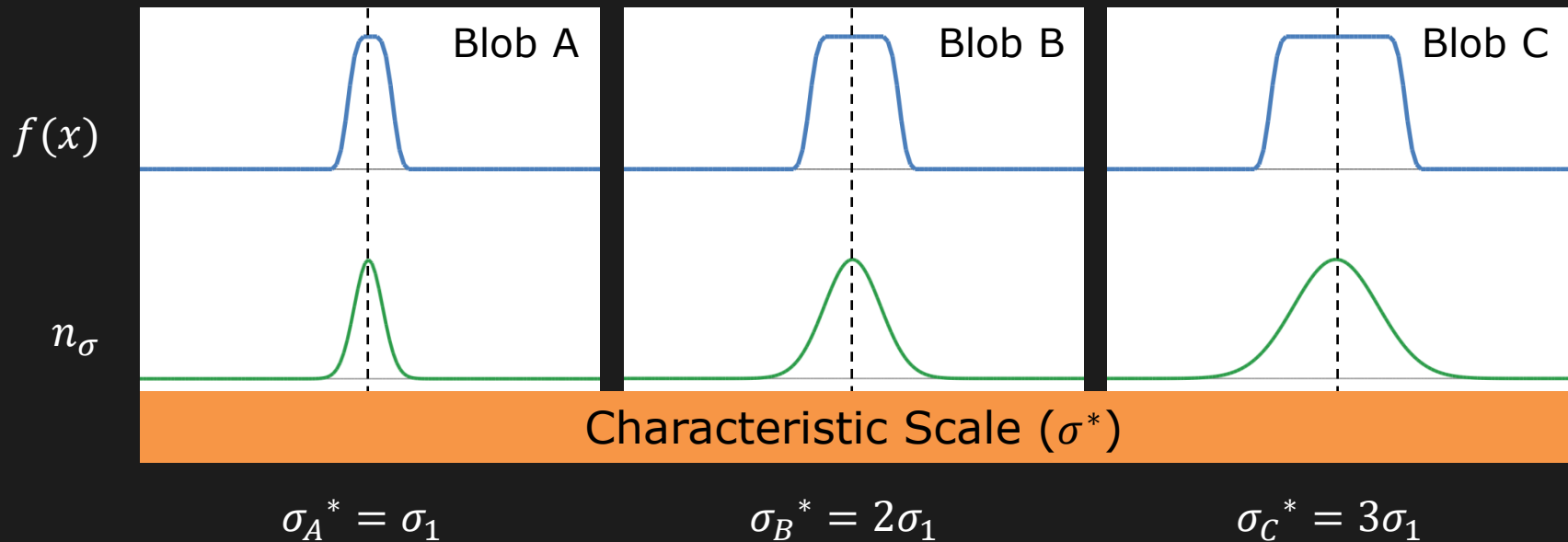
1D Blob and 2nd Derivative of Gaussian



1D Blob and 2nd Derivative of Gaussian



Characteristic Scale and Blob Size



Characteristic Scale: The σ at which 2nd Derivative attains its extreme value.

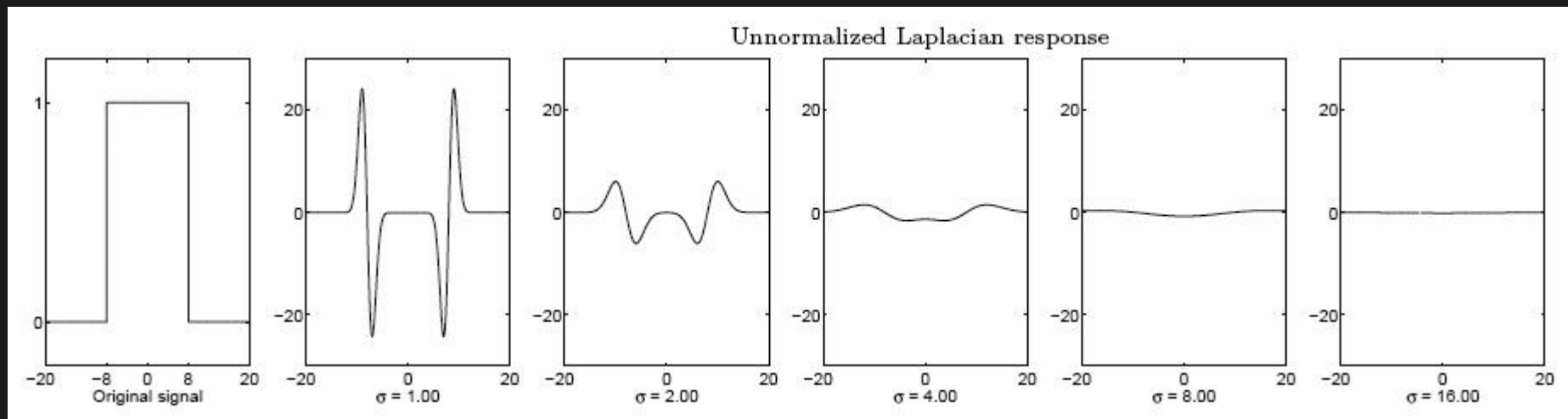
Characteristic Scale $\propto$ Size of Blob



$$\frac{\text{Size of Blob A}}{\text{Size of Blob B}} = \frac{\sigma_A^*}{\sigma_B^*} ; \quad \frac{\text{Size of Blob B}}{\text{Size of Blob C}} = \frac{\sigma_B^*}{\sigma_C^*}$$

Scale Selection

- We want to find the characteristic scale of the blob by convolving it with Laplacians at several scales and looking for the maximum response
- However, Laplacian response decays as scale increases:



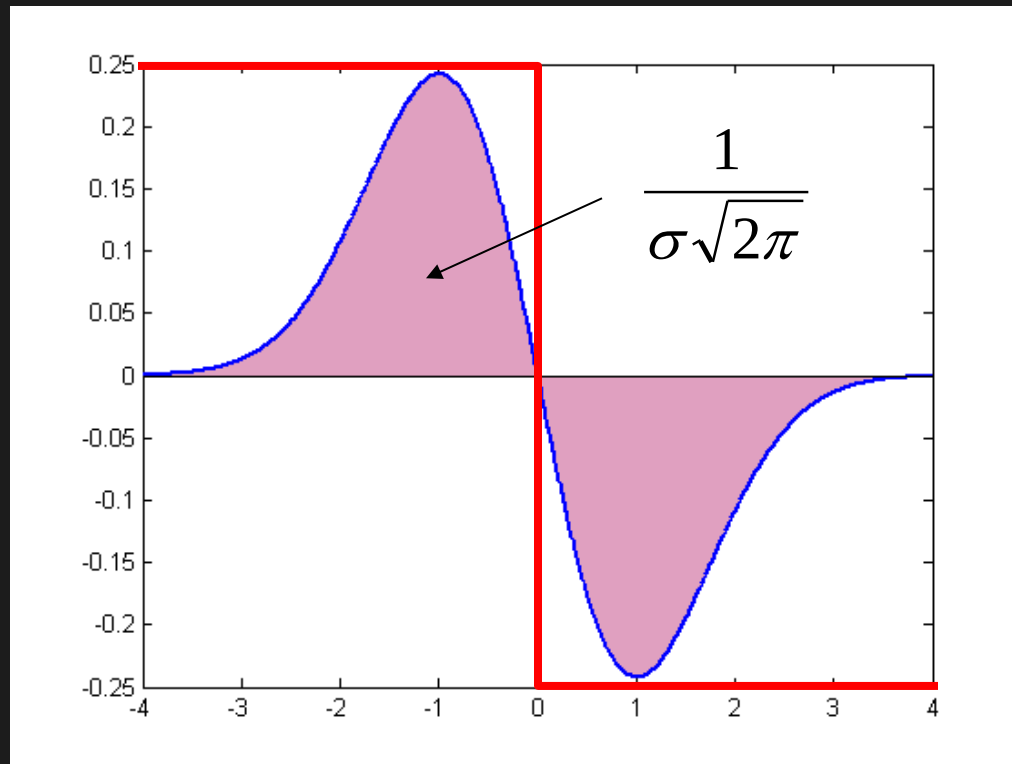
original signal
(radius=8)

increasing σ $\longrightarrow$

Why does this happen?

Scale Normalization

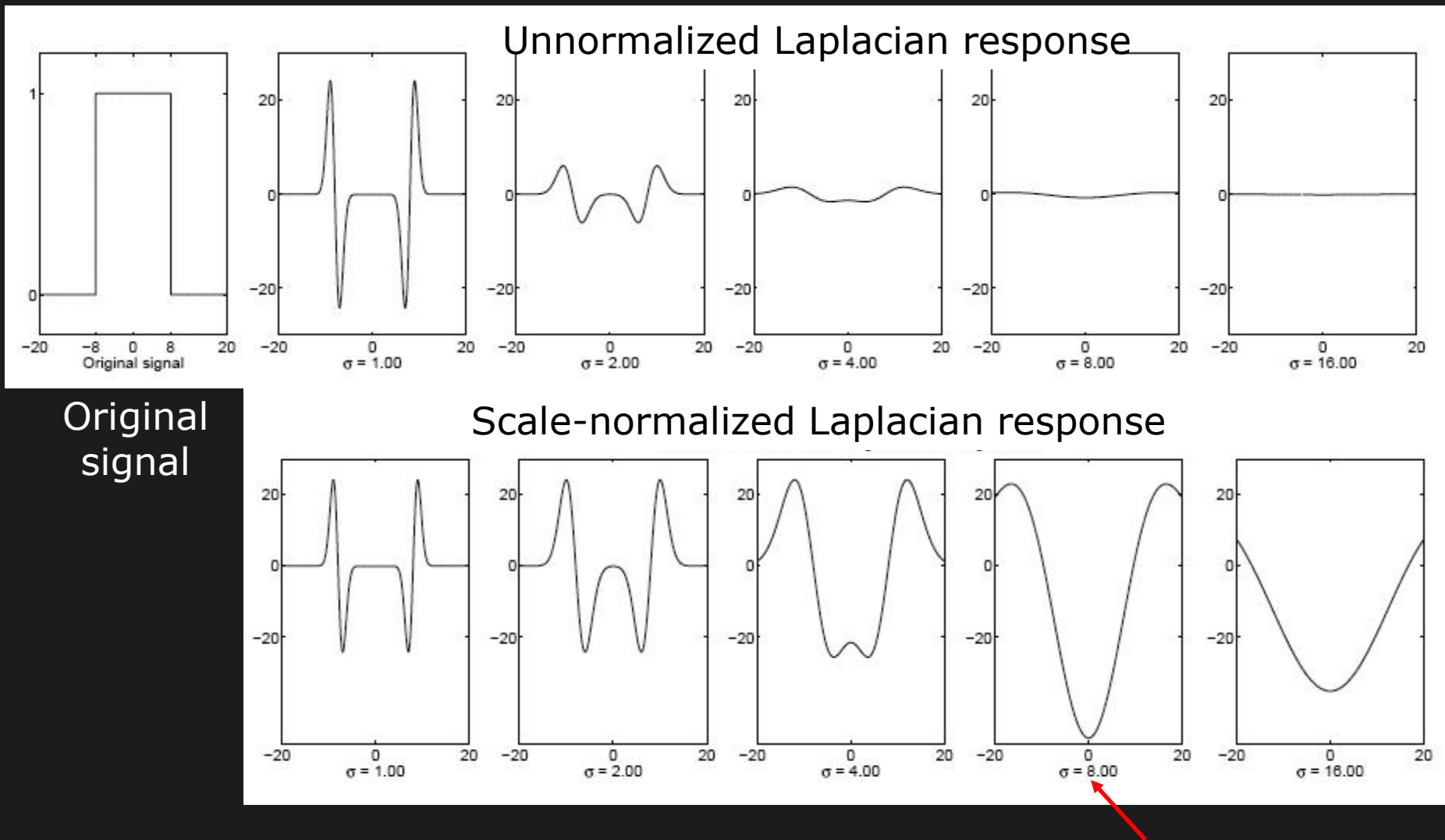
- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases



Scale Normalization

- The response of a derivative of Gaussian filter to a perfect step edge decreases as σ increases
- To keep response the same (scale-invariant), must multiply Gaussian derivative by σ
- Laplacian is the second Gaussian derivative, so it must be multiplied by σ^2

Effect of Scale Normalization



1D Blob Detection Summary

Given a 1D signal $f(x)$.

Compute $\frac{\partial^2 n_\sigma}{\partial x^2} * f(x)$ at many scales $(\sigma_0, \sigma_1, \sigma_2, \dots, \sigma_k)$.

Find:

$$(x^*, \sigma^*) = \arg \max_{(x, \sigma)} \left| \frac{\partial^2 n_\sigma}{\partial x^2} * f(x) \right|$$

x^* : Blob Position

σ^* : Characteristic Scale (Blob Size)

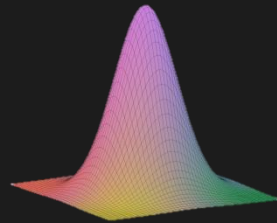
2D Blob Detector

Normalized Laplacian of Gaussian (NLoG) is used as the 2D equivalent for Blob Detection.

Laplacian

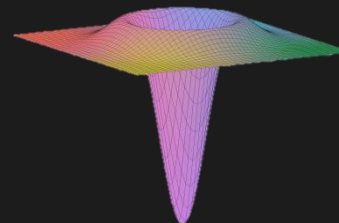
$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

Gaussian



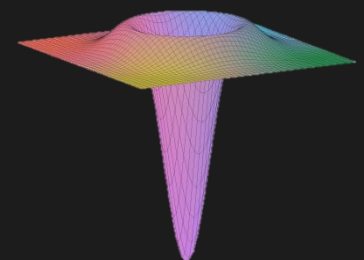
n_σ

LoG



$\nabla^2 n_\sigma$

NLoG



$\sigma^2 \nabla^2 n_\sigma$

Location of Blobs given by **Local Extrema** after applying Normalized Laplacian of Gaussian at many scales.

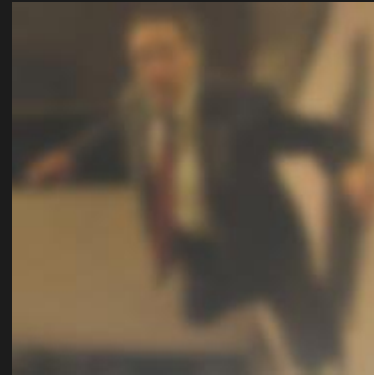
Scale-Space



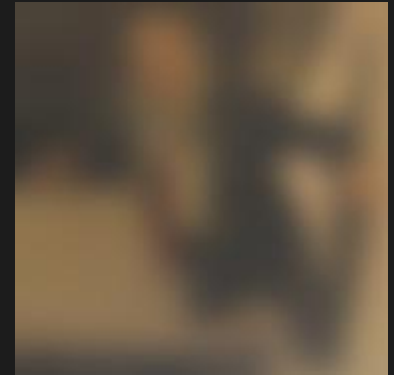
$S(x, y, \sigma_0)$



$S(x, y, \sigma_1)$



$S(x, y, \sigma_2)$



$S(x, y, \sigma_3)$

...

Increasing σ , Higher Scale, Lower Resolution

Scale Space: Stack created by filtering an image with Gaussians of different sigma(σ)

$$S(x, y, \sigma) = n(x, y, \sigma) * I(x, y)$$

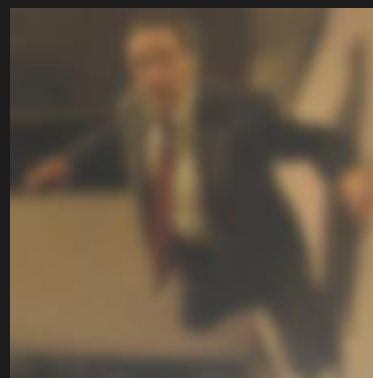
Creating Scale-Space



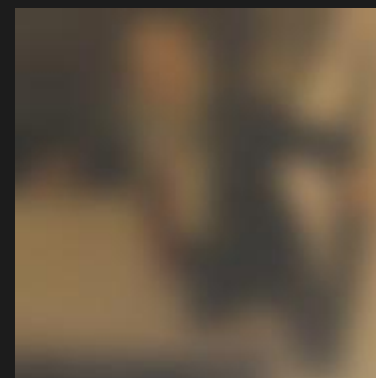
$S(x, y, \sigma_0)$

s : Constant multiplier

$S(x, y, \sigma_1)$



$S(x, y, \sigma_2)$



$S(x, y, \sigma_3)$

...

Increasing σ , Higher Scale, Lower Resolution

Selecting sigma's to generate the scale-space:

$$\sigma_k = \sigma_0 s^k \quad k = 0, 1, 2, 3, \dots$$

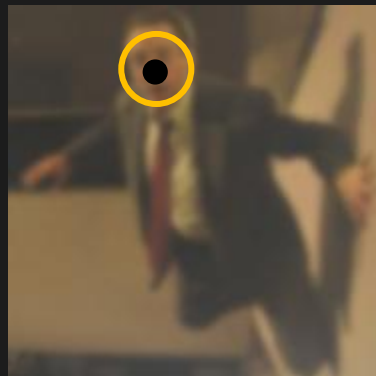
where, s : Constant multiplier

σ_0 : Initial Scale

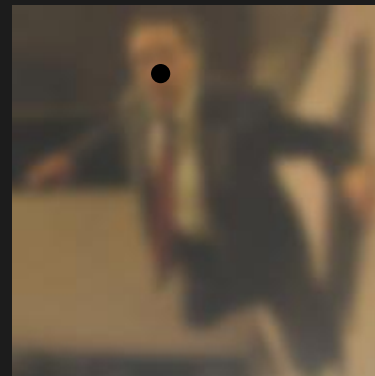
Blob Detection using Local Extrema



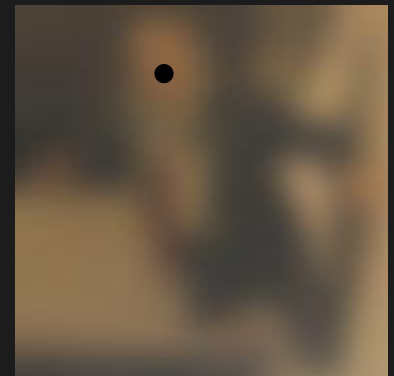
$S(x, y, \sigma_0)$



$S(x, y, \sigma_1)$



$S(x, y, \sigma_2)$

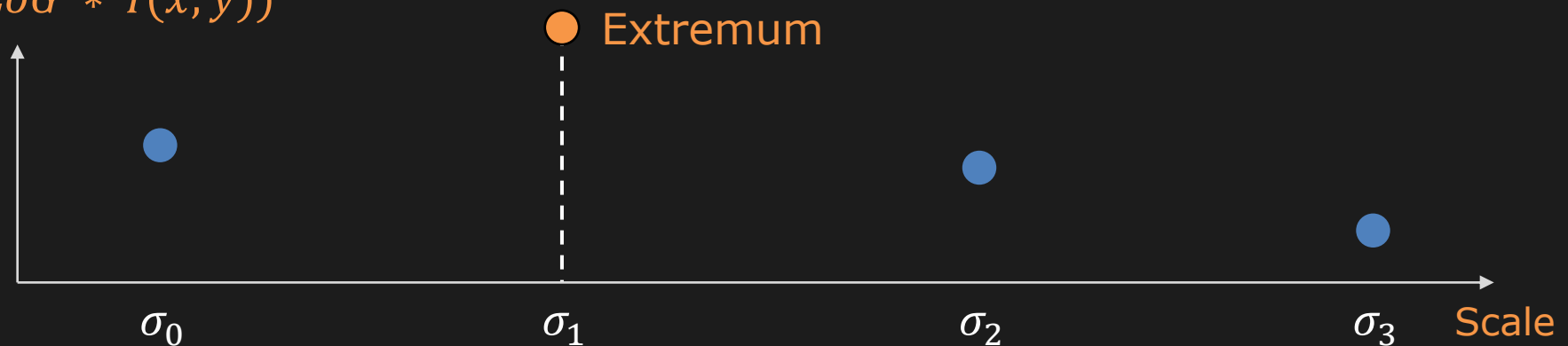


$S(x, y, \sigma_3)$

...

$$\sigma^2 \nabla^2 S(x, y, \sigma)$$

(NLoG * I(x, y))



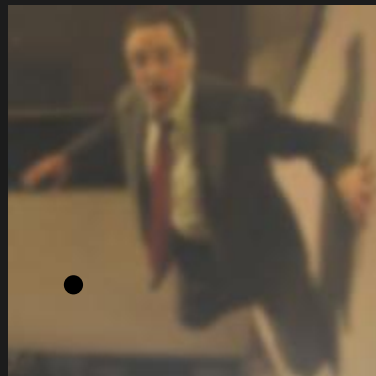
Characteristic Scale (σ^*)

[Lindeberg 1994]

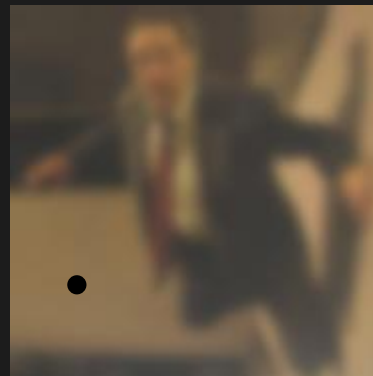
Blob Detection using Local Extrema



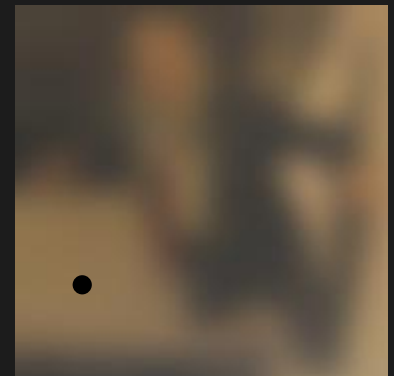
$S(x, y, \sigma_0)$



$S(x, y, \sigma_1)$



$S(x, y, \sigma_2)$

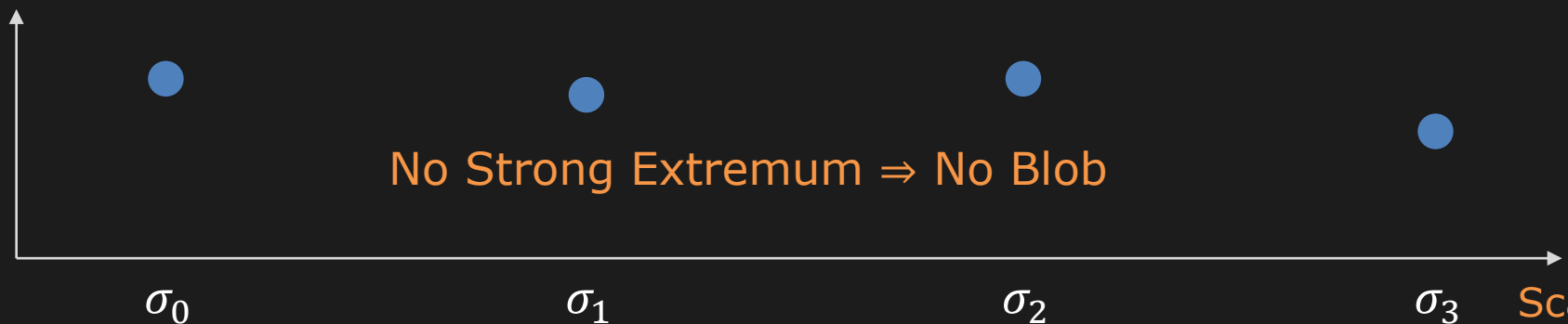


$S(x, y, \sigma_3)$

...

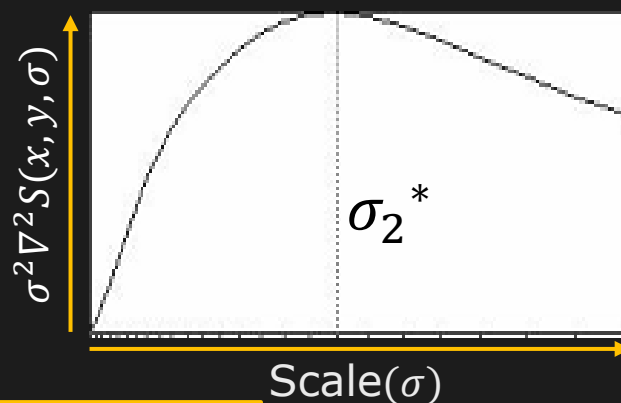
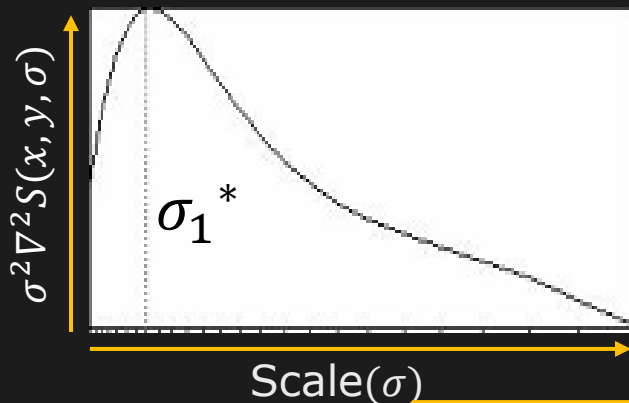
$$\sigma^2 \nabla^2 S(x, y, \sigma)$$

(*NLoG* * $I(x, y)$)



[Lindeberg 1994]

Comparison of Characteristic Scales



$\frac{\sigma_1^*}{\sigma_2^*}$: Ratio of Blob Sizes

[Mikołajczyk 2002]

2D Blob Detection Summary

Given an image $I(x, y)$.

Convolve the image using NLoG at many scales σ .

Find: {

$$(x^*, y^*, \sigma^*) = \arg \max_{(x, y, \sigma)} |\sigma^2 \nabla^2 n_\sigma * I(x, y)|$$

or

$$(x^*, y^*, \sigma^*) = \arg \max_{(x, y, \sigma)} |\sigma^2 \nabla^2 S(x, y, \sigma)|$$

(x^*, y^*) : Position of the blob

σ^* : Size of the blob

Scale-invariant Feature Transform (SIFT)

An Efficient Implementation of Blob Detector



David Lowe

Professor

University of British Columbia



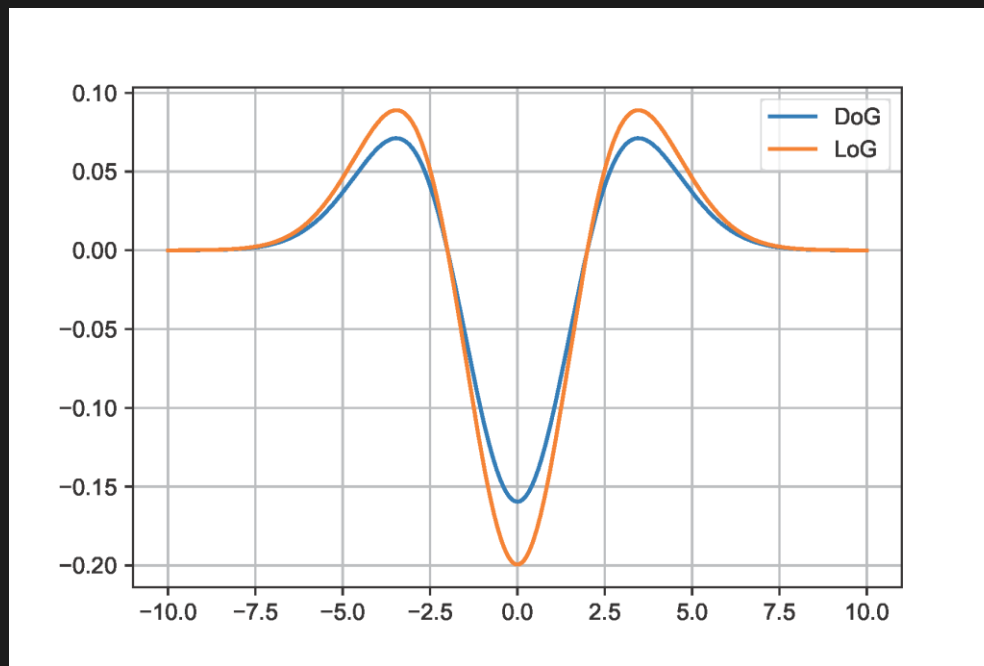
Cloudburst Research



The SIFT Detector

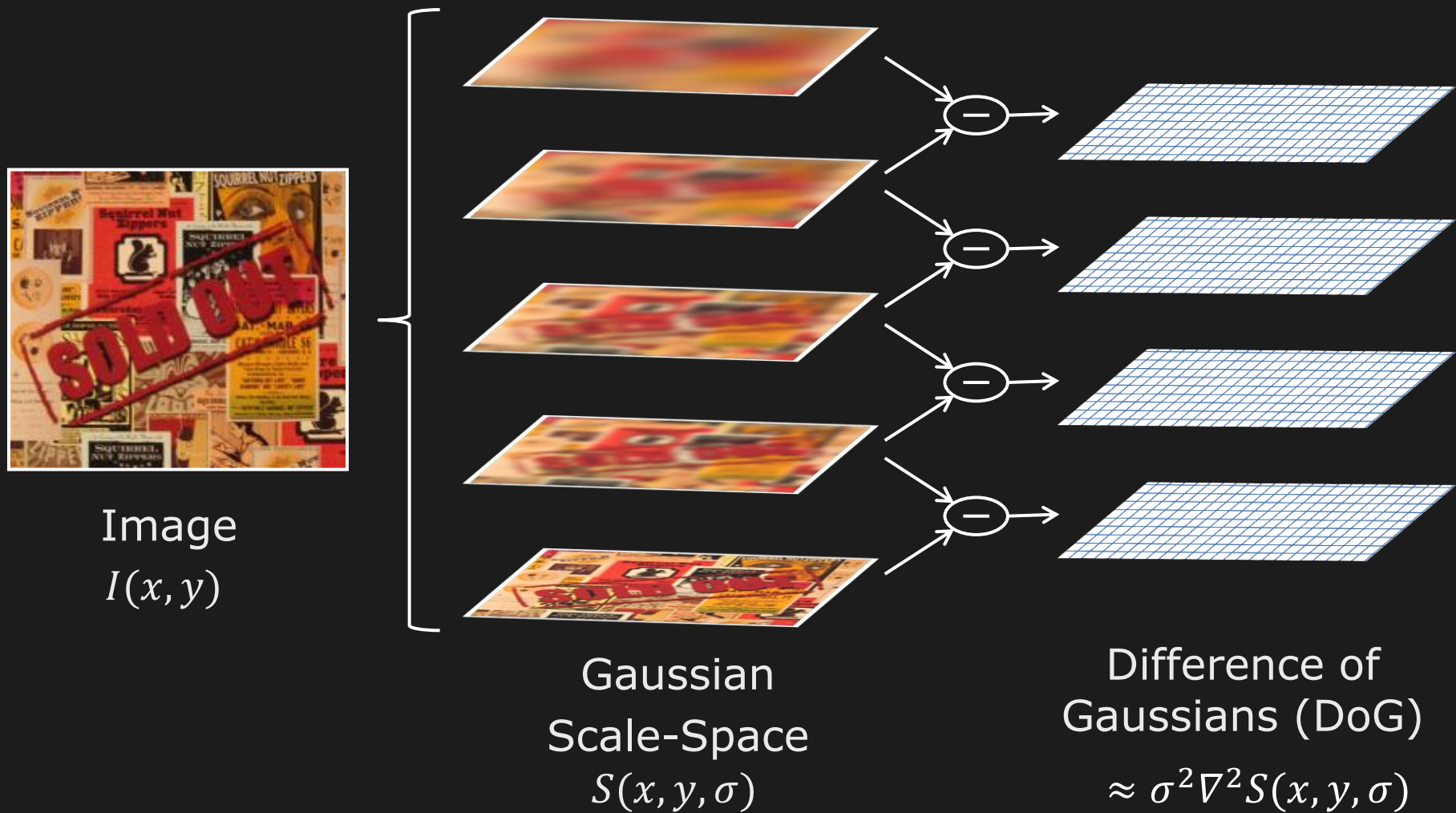
Uses **Difference of Gaussian (DoG)** as an approximation of NLoG

$$\text{Difference of Gaussian(DoG)} = (n_{s\sigma} - n_{\sigma}) \approx (s - 1)\sigma^2 \nabla^2 n_{\sigma}$$

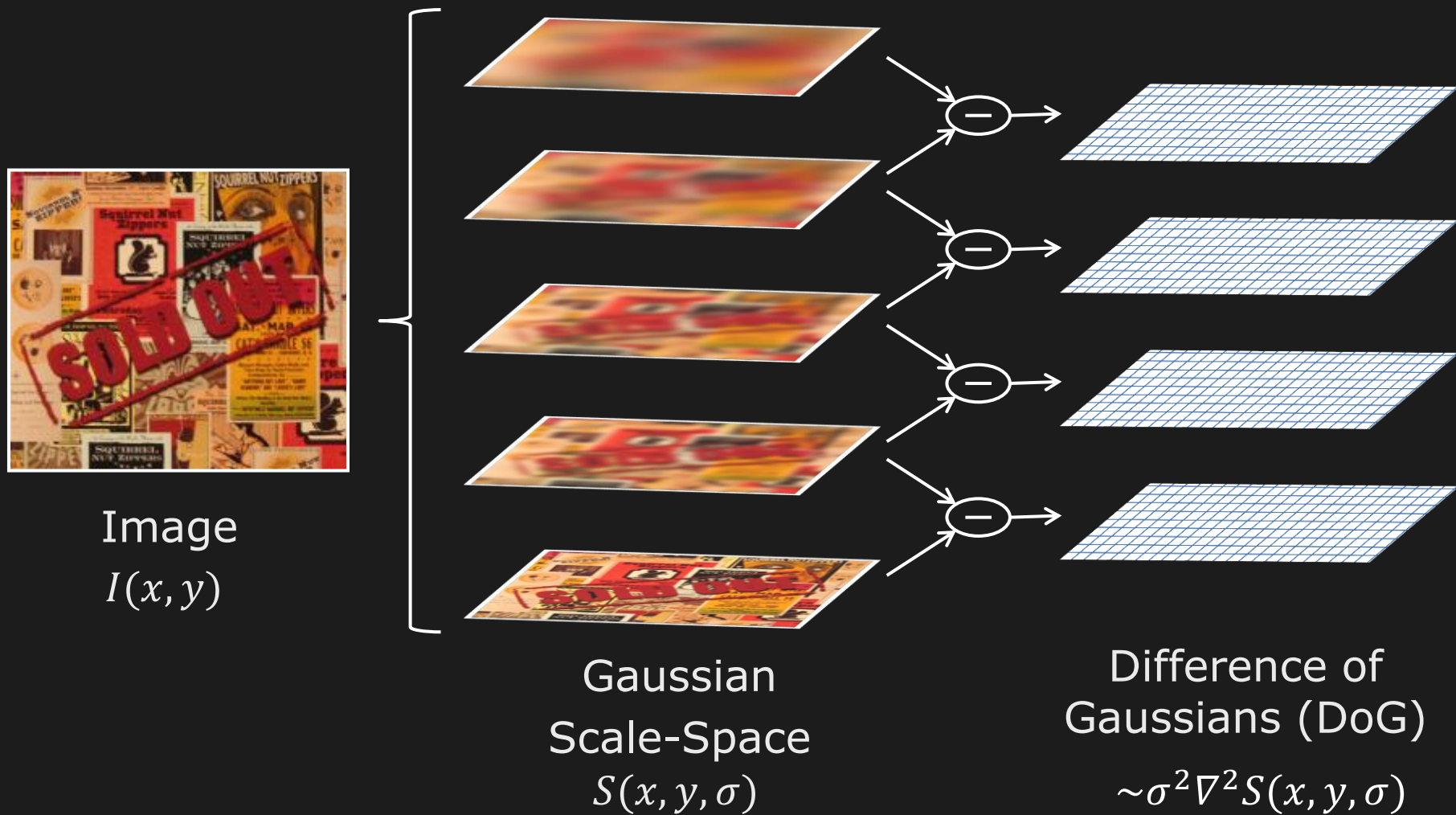


DoG $\approx$ NLoG

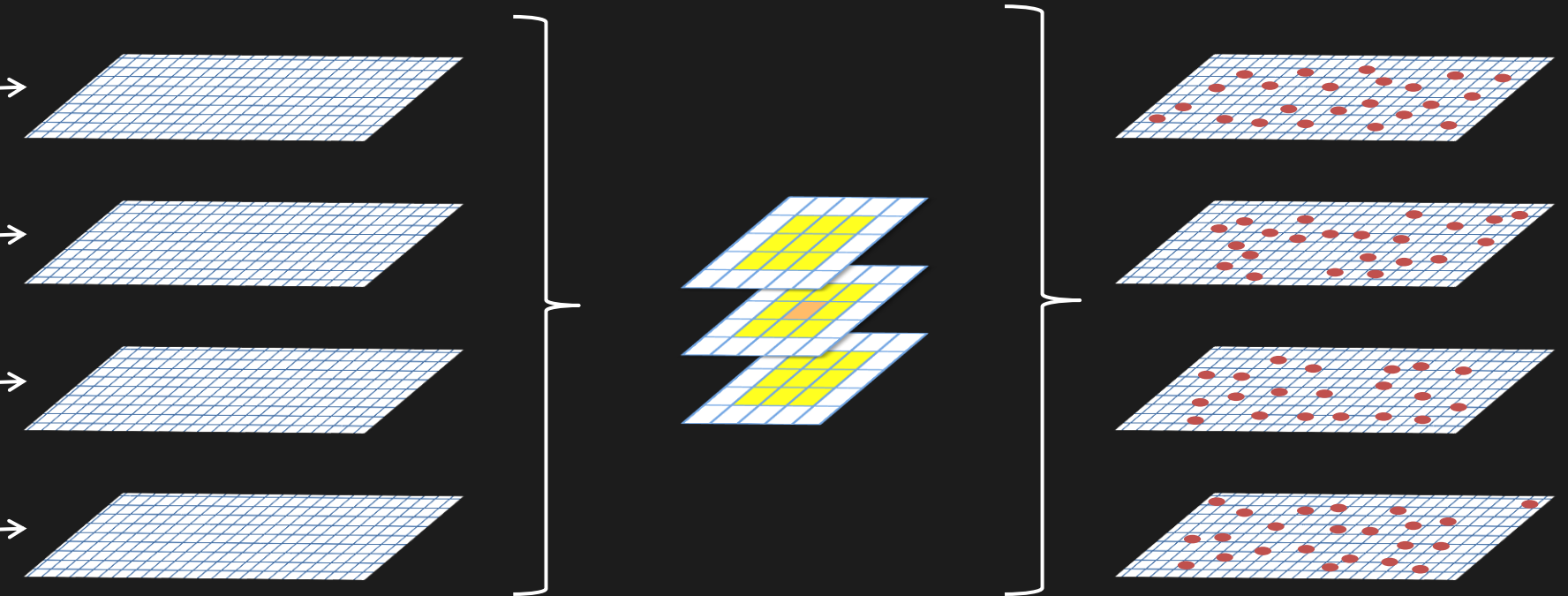
Extracting SIFT Interest Points



Extracting SIFT Interest Points



Extracting SIFT Interest Points



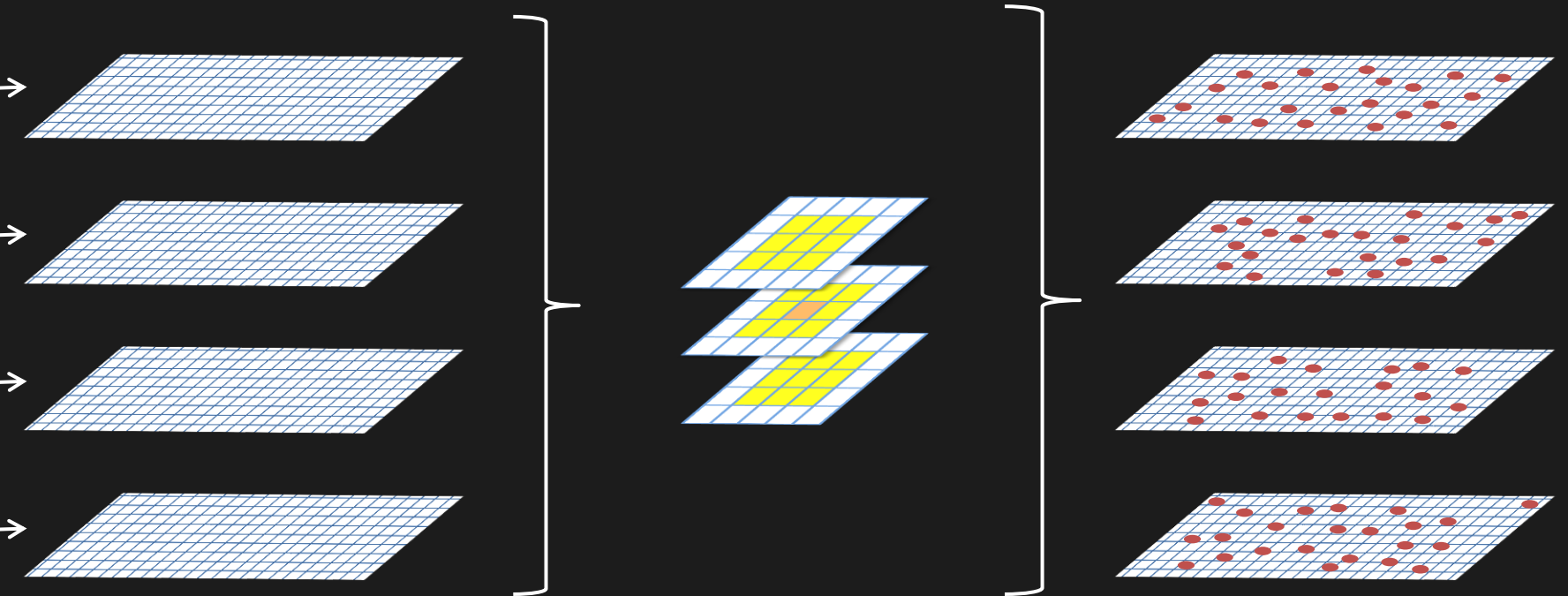
Difference of
Gaussians (DoG)

$$\sim \sigma^2 \nabla^2 S(x, y, \sigma)$$

Find Extrema
in every
3x3x3 grid

Interest Point
Candidates
(includes weak extrema,
bad contrast, ...)

Extracting SIFT Interest Points



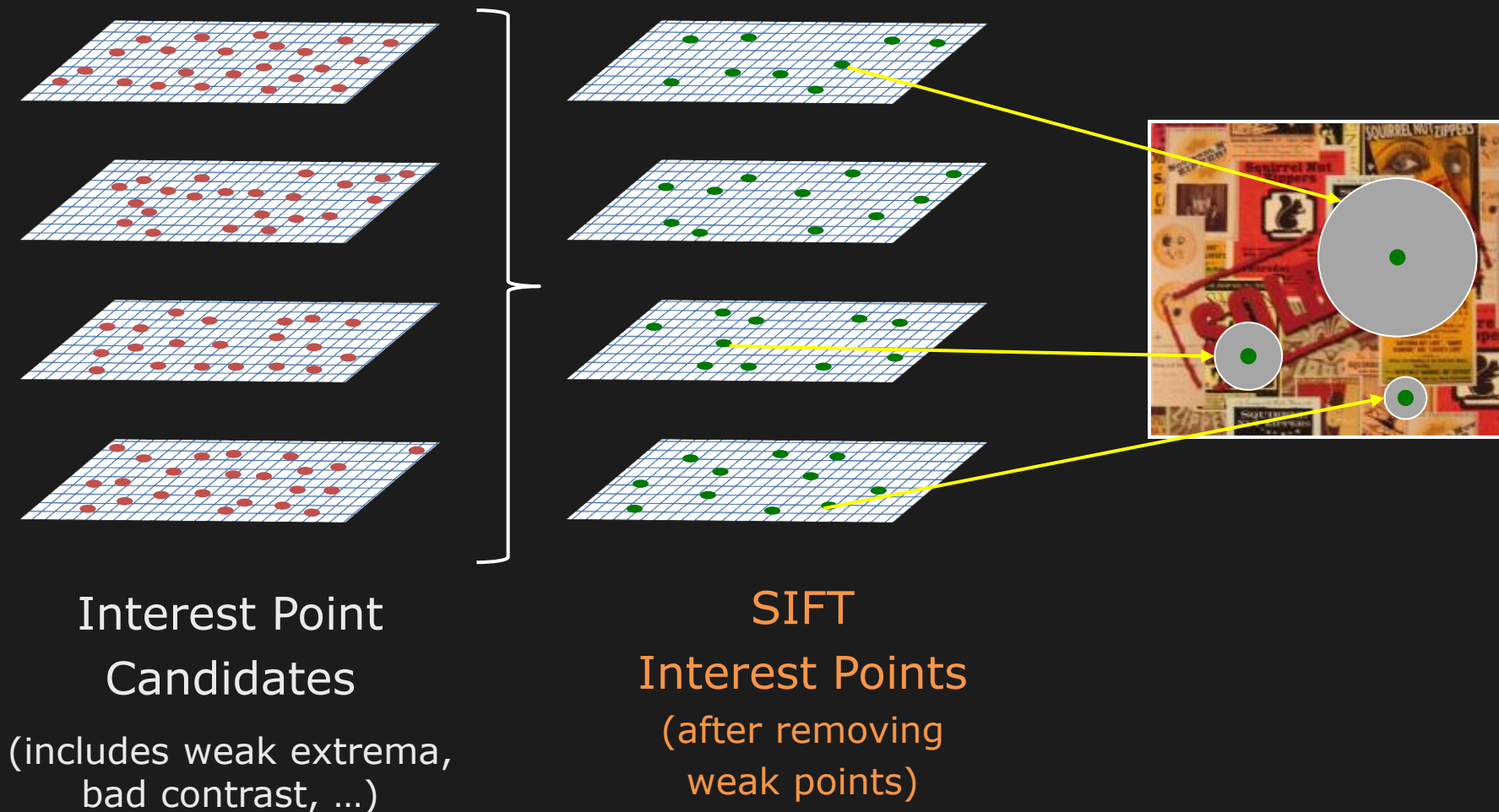
Difference of
Gaussians (DoG)

$$\sim \sigma^2 \nabla^2 S(x, y, \sigma)$$

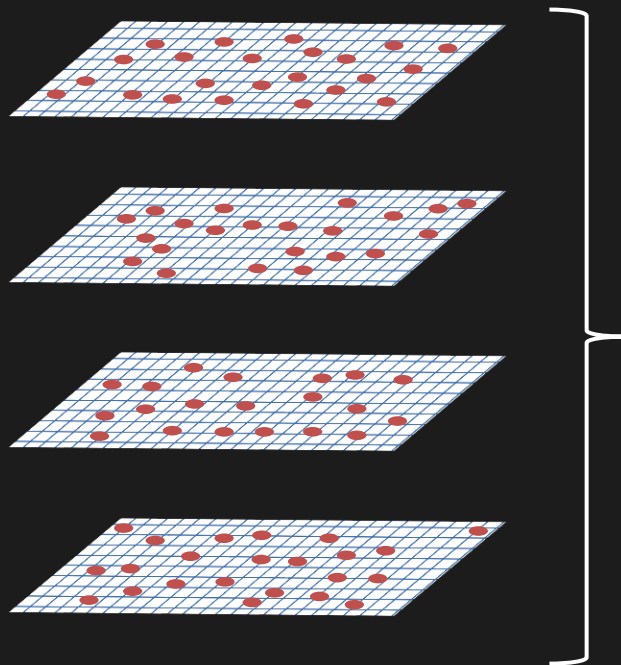
Find Extrema
in every
3x3x3 grid

Interest Point
Candidates
(includes weak extrema,
bad contrast, ...)

Extracting SIFT Interest Points

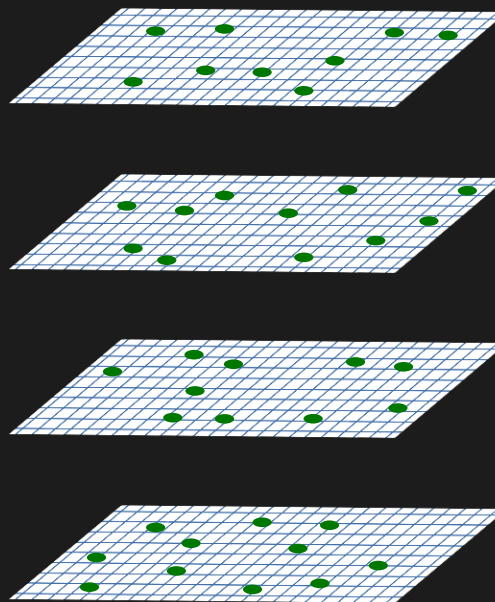


Extracting SIFT Interest Points



Interest Point
Candidates

(includes weak extrema,
bad contrast, ...)



SIFT
Interest Points
(after removing
weak points)



Interest Point
Depiction

SIFT Detection Examples



SIFT Detection Examples



SIFT Detection Examples



SIFT Scale Invariance

Use the characteristic scales to match sizes



Computing the Principal Orientation

Use the histogram of gradient directions

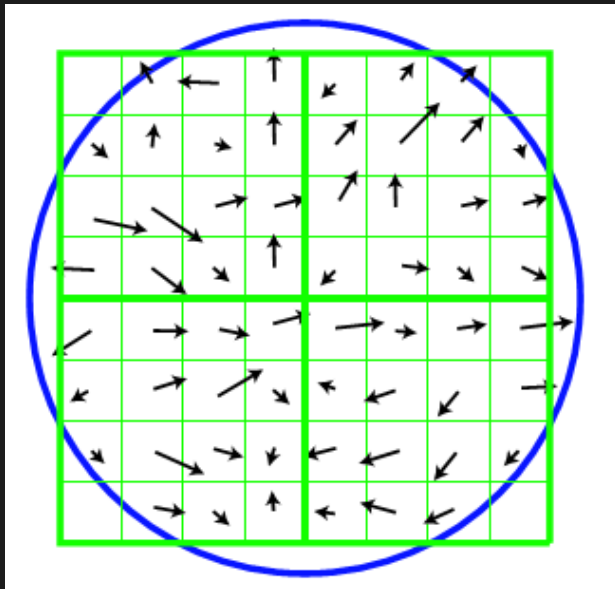
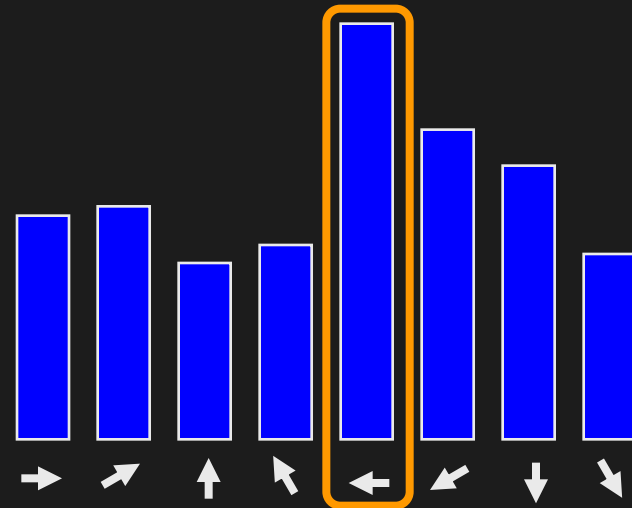


Image gradient directions

$$\theta = \tan^{-1} \left(\frac{\partial I}{\partial y} / \frac{\partial I}{\partial x} \right)$$

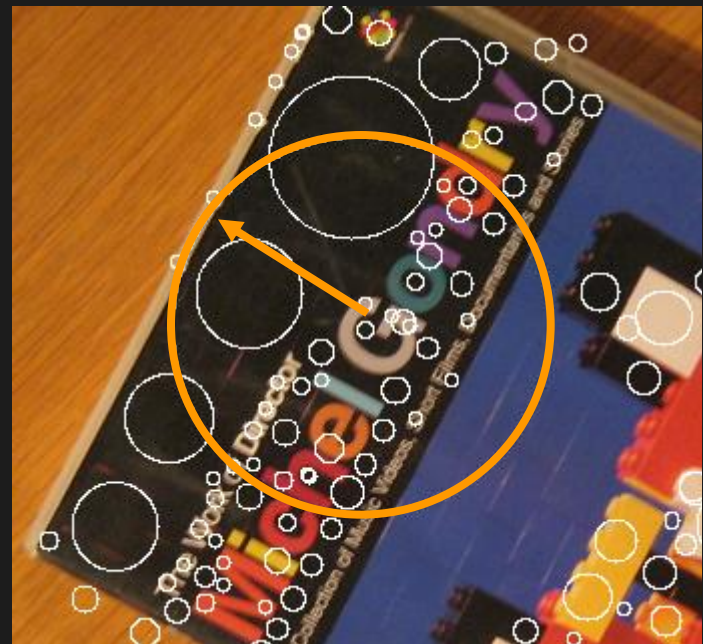
Principal Orientation



Choose the most prominent gradient direction

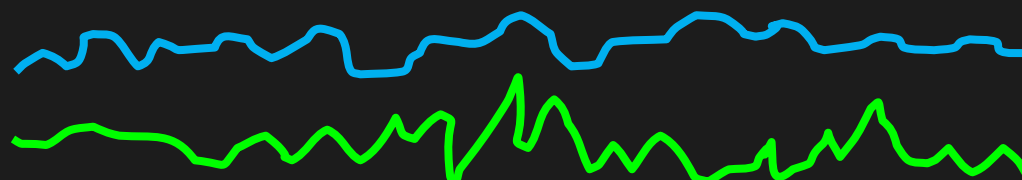
SIFT Rotation Invariance

Use the principal orientations to match rotation



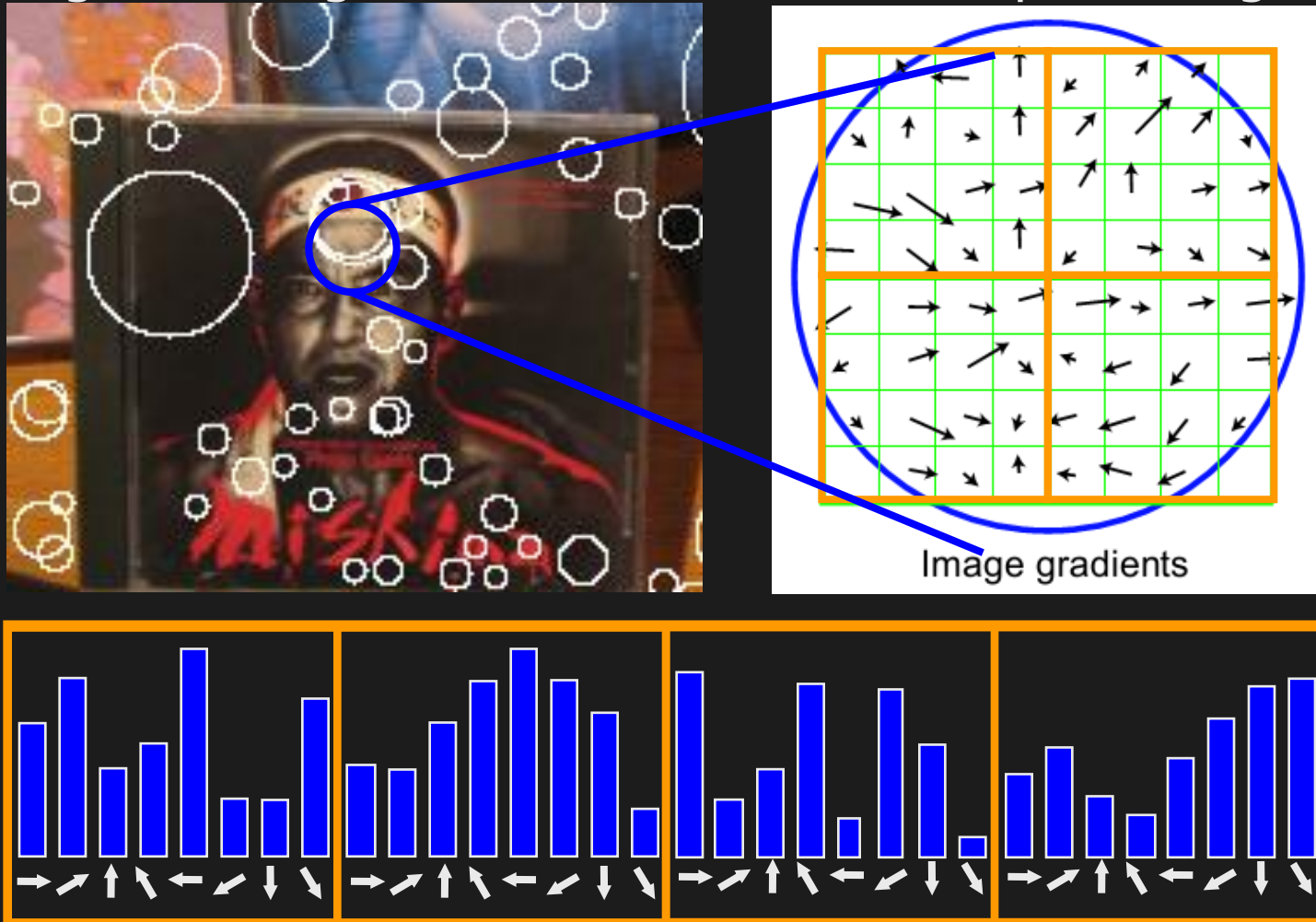
The SIFT Descriptor

“Describe” points so they can be compared



Computing the SIFT Descriptor

Histograms of gradient directions over spatial regions



Invariant to Scale, Lighting, Brightness

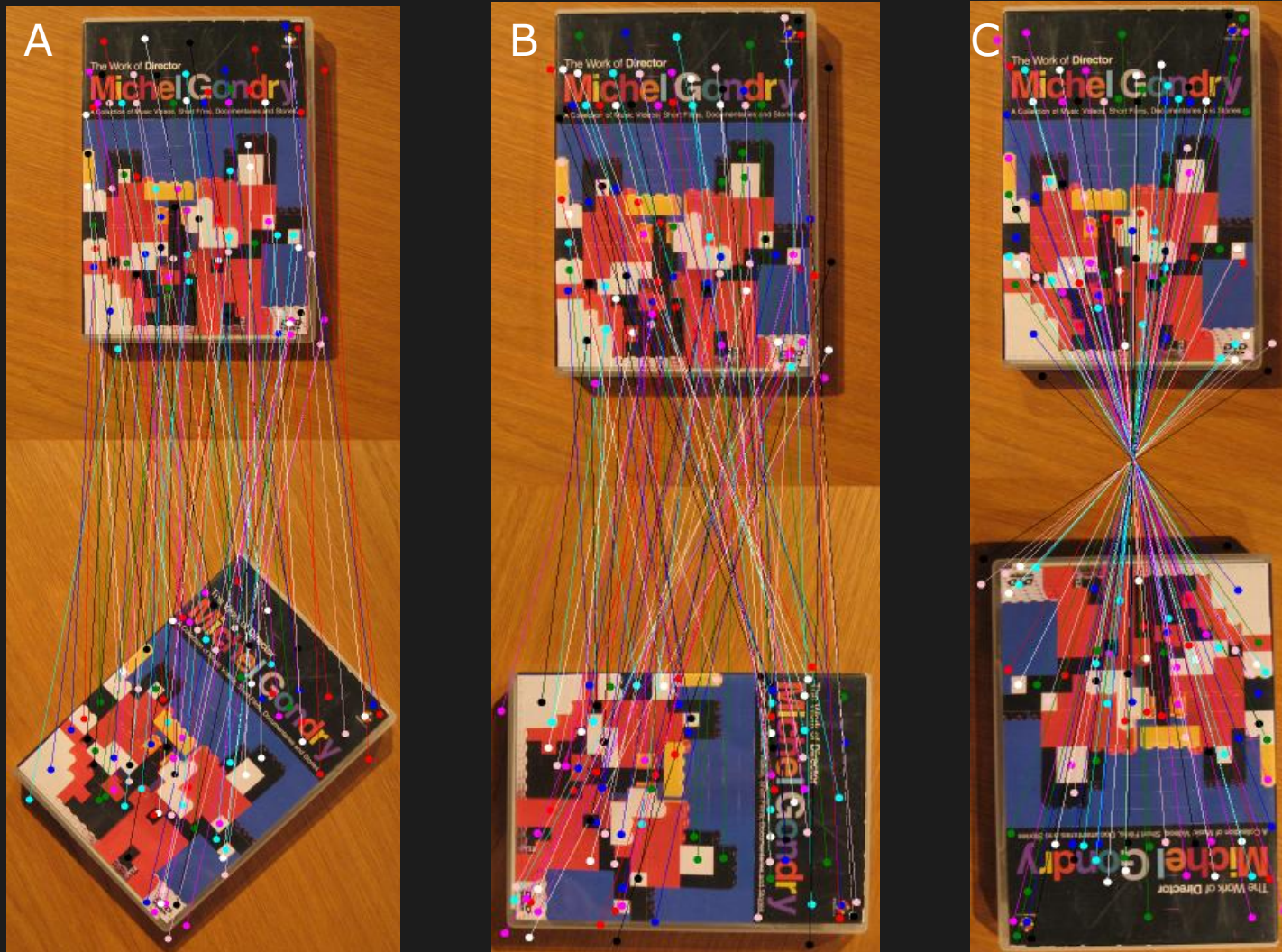
Computer Vision, SJTU, Wei Shen

SIFT Results: Scale Invariance



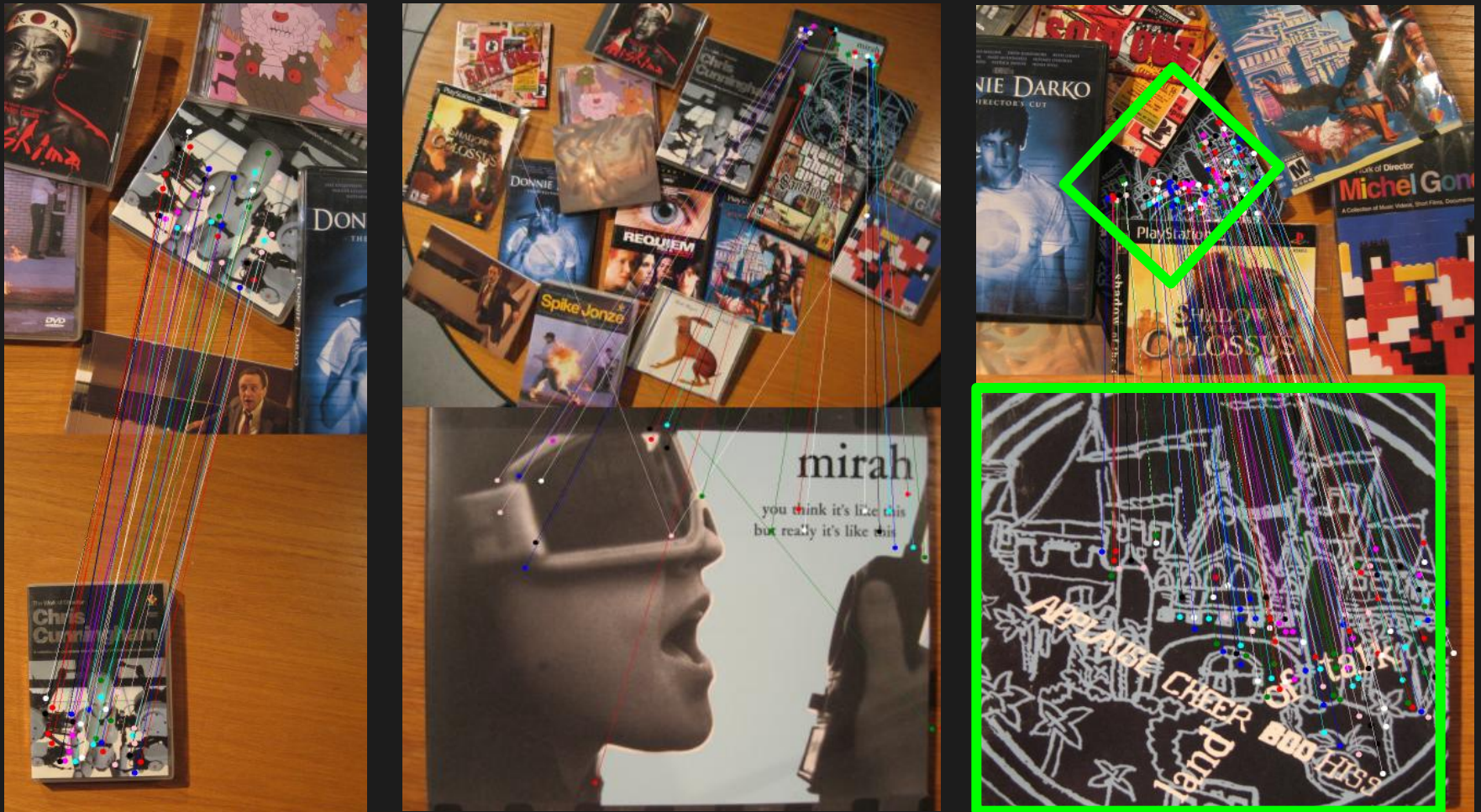
SIFT detects corresponding features in images at different resolutions

SIFT Results: Rotation Invariance



SIFT detects corresponding features in rotated images

SIFT Robustness to Clutter



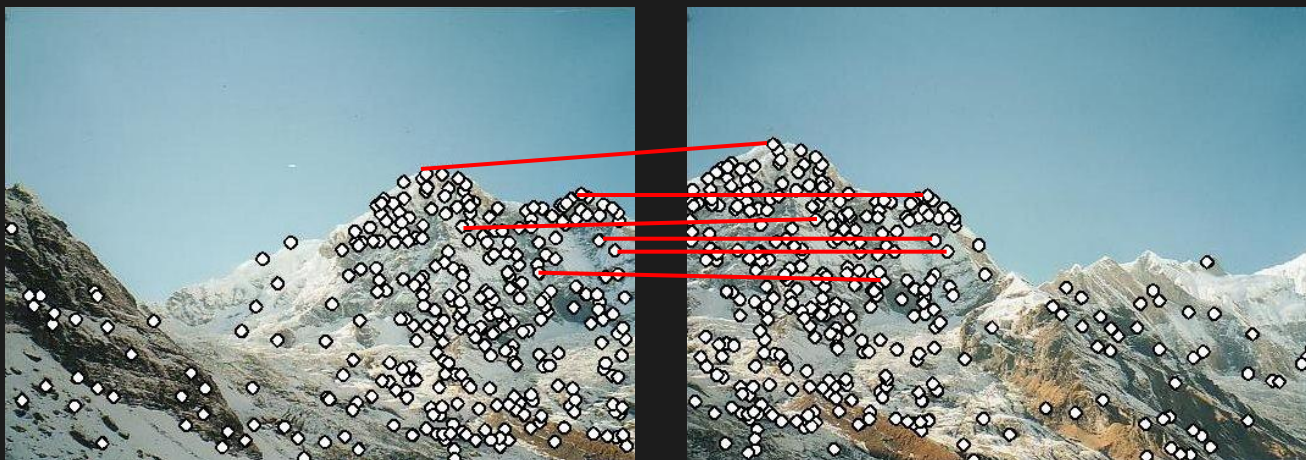
Panorama Stitching using SIFT



Image 1



Image 2



Match SIFT Interest Points
Computer Vision, SJTU, Wei Shen

Panorama Stitching using SIFT



Transform/Warp one or both images so that corresponding SIFT points in images are aligned.

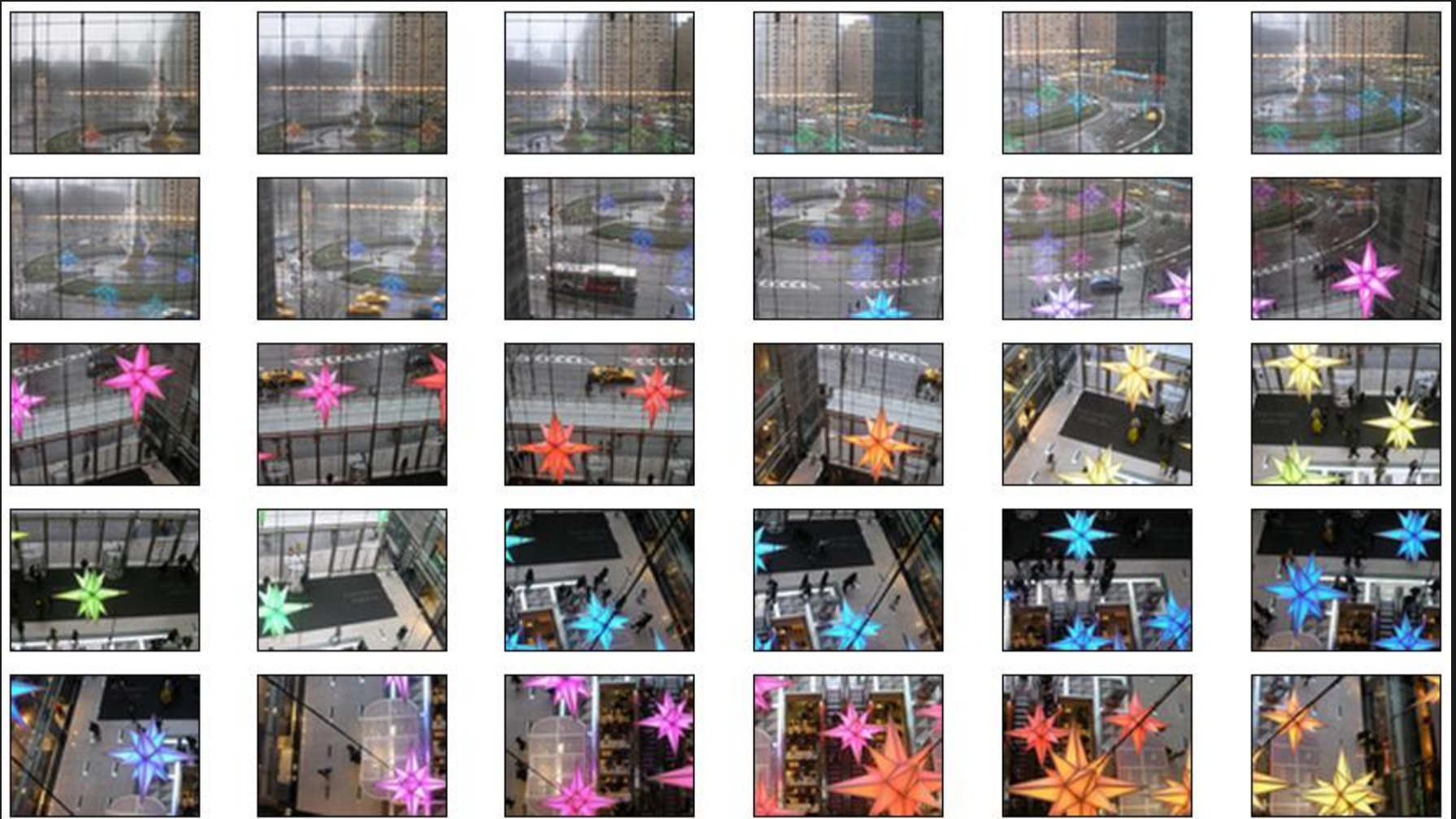
Panorama Stitching using SIFT

AutoStitch:

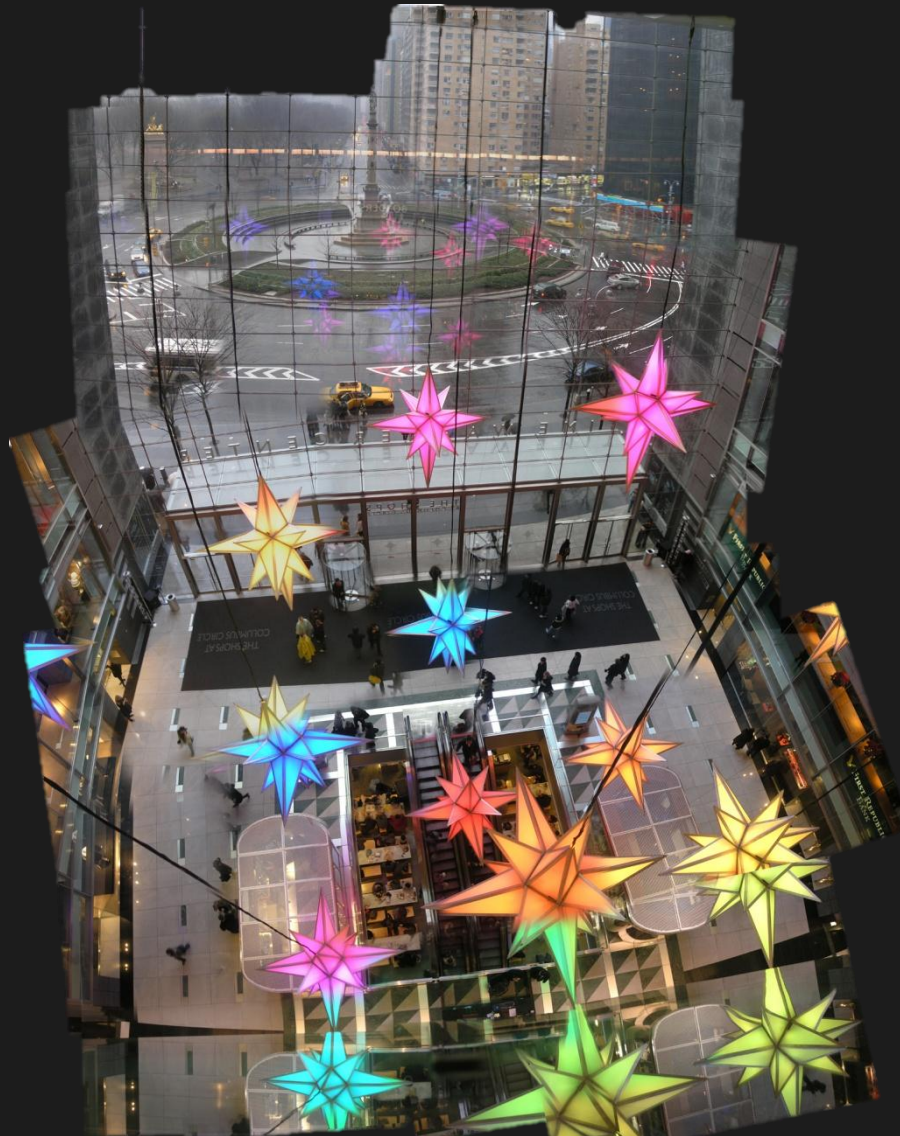
<http://www.cs.bath.ac.uk/brown/autostitch/autostitch.html>

(More on this in Image Stitching Lecture)

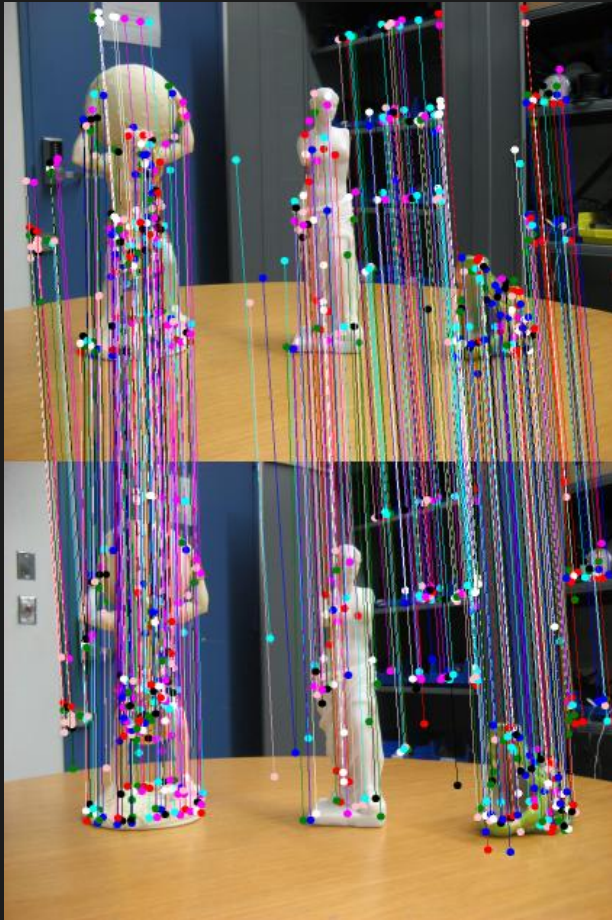
Auto Collage using SIFT



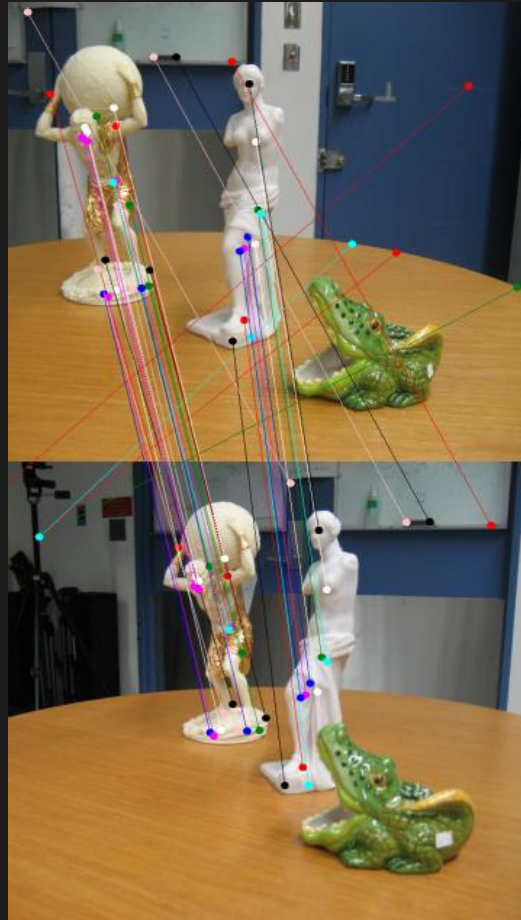
Auto Collage using SIFT



SIFT for 3D Objects?



No Change in Viewpoint



30° Change in Viewpoint



90° Change in Viewpoint

Other Descriptors

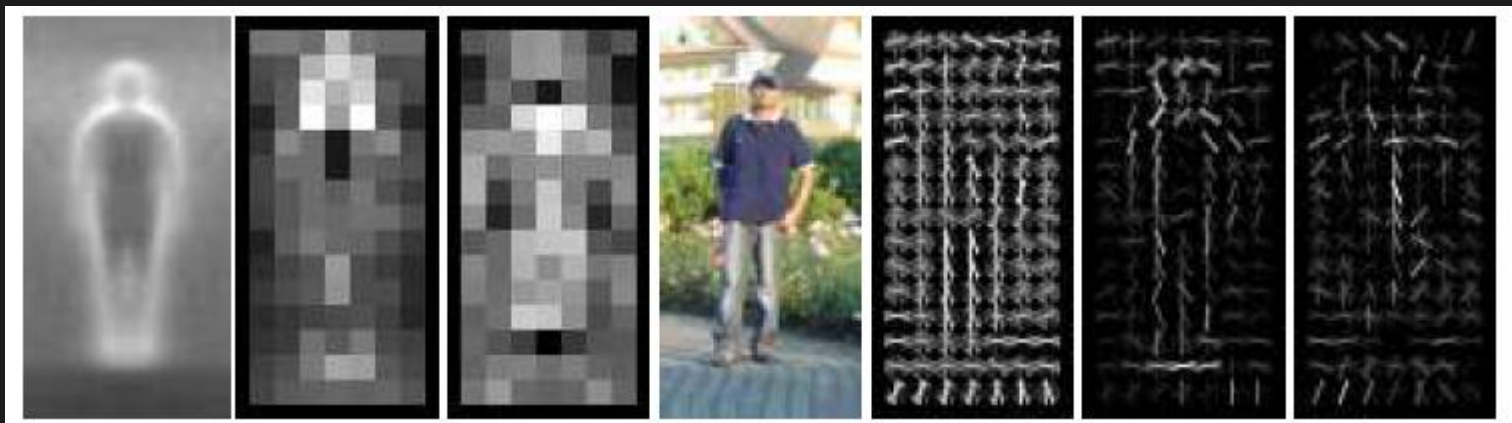
Distinguish between Feature *Detector* and Feature *Descriptor*

The ability to match features robustly lies in good feature descriptors!

Histogram-approaches are popular because of: compactness, linearity, uniqueness (think clustering), always same size regardless of image patch size!

Histogram-of-Oriented-Gradients

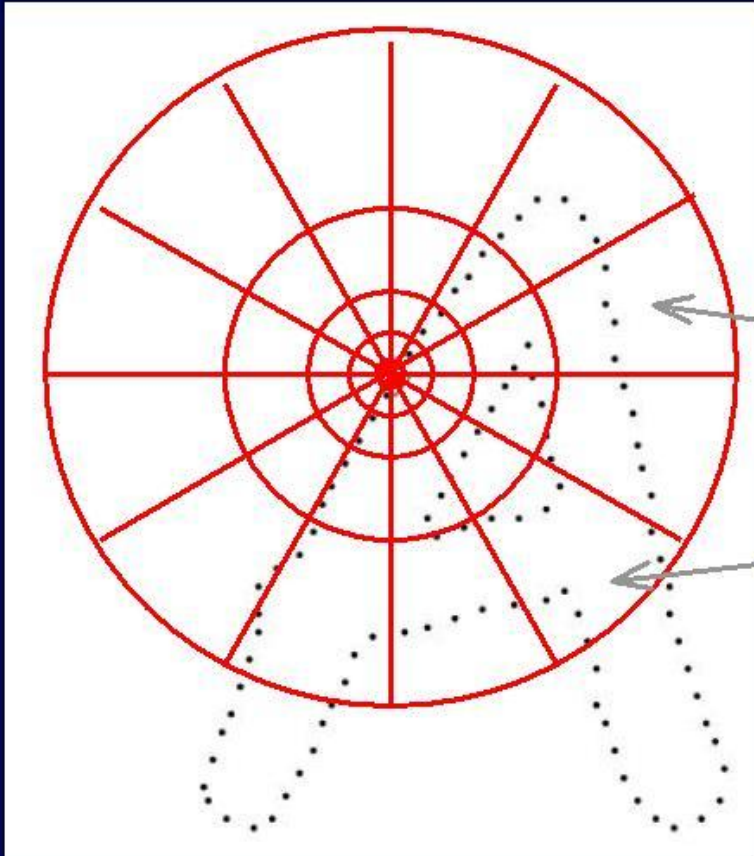
HoG (Dalal, et. al., CVPR 2005) another similar approach: made popular to detect pedestrians in images



Shape Contexts

Shape Contexts (Belongie, et. al. 2002): descriptor to match *shapes*. Demonstrated on handwriting recognition

Log Polar
Distribution
(edge
points)



Count the number of points
inside each bin, e.g.:

Count = 4

⋮

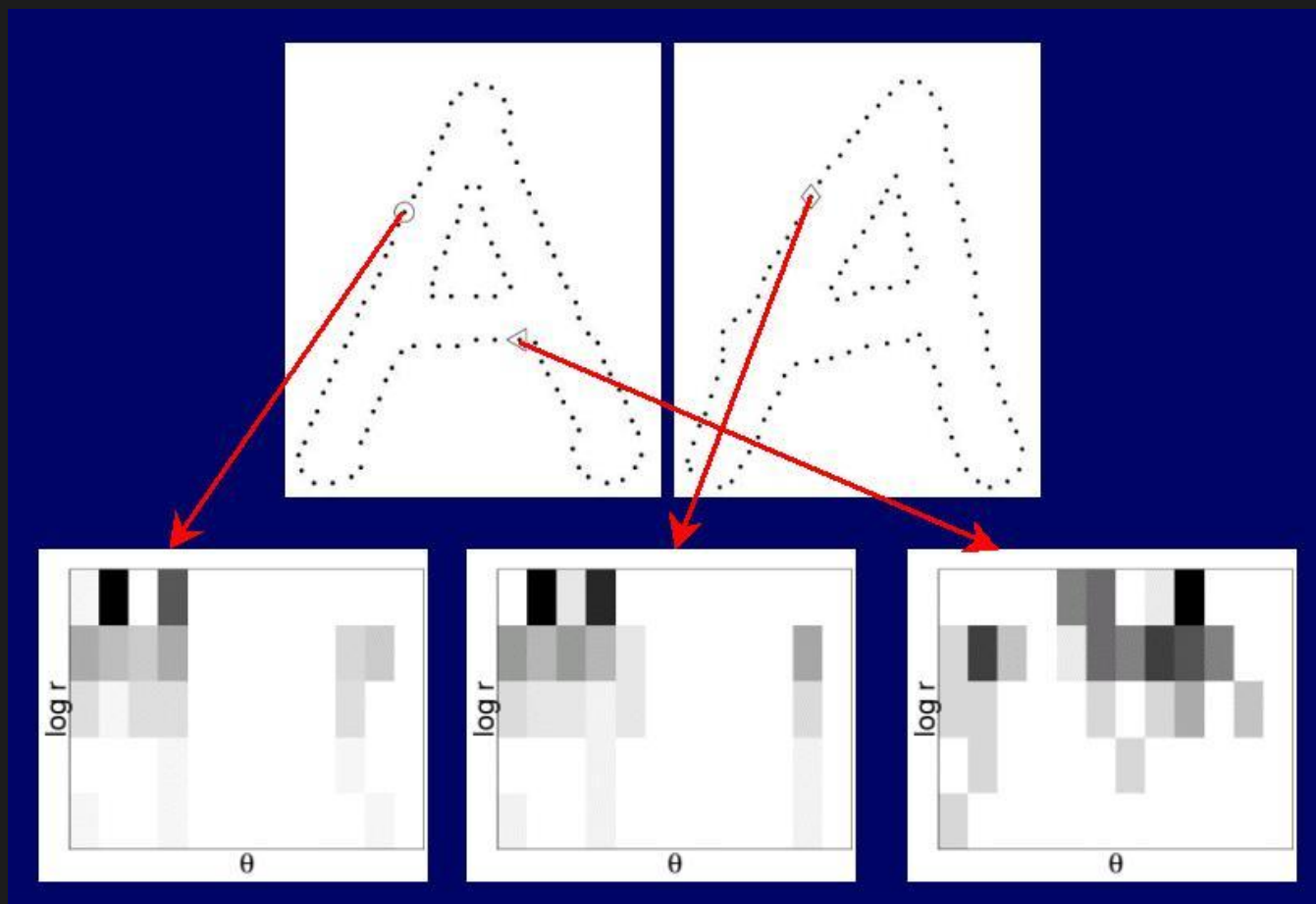
Count = 10

☞ Compact representation
of distribution of points
relative to each point

Shape Contexts

Shape Contexts (Belongie, et. al. 2002): descriptor to match *shapes*. Demonstrated on handwriting recognition

Log Polar
Distribution
(edge
points)



References

[Autopano] Software to make panoramas using SIFT. <http://user.cs.tu-berlin.de/~nowozin/autopano-sift/>

[Brown and Lowe 2002] M. Brown and D. Lowe. "Invariant Features from Interest Point Groups". *BMVC*, 2002.

[Harris and Stephens 1988] C. Harris and M. Stephens. "A Combined Corner and Edge Detector". *4th Alvey Vision Conference*, 1988.

[Lowe 2004] D. Lowe. "Distinctive Image Features from Scale-Invariant Keypoints". *IJCV*, 2004.

[Lindeberg 1994] T. Lindeberg. "Scale-Space Theory: A Basic Tool for Analysing Structures at Different Scales." *J. of Applied Statistics*, 1994.

[Matas 2002] J. Matas, O. Chum, M. Urban, and T. Pajdla. "Robust Wide Baseline Stereo from Maximally Stable Extremal Regions. *BMVC*, 2002.

[Mikolajczyk 2002] K. Mikolajczyk. "Detection of Local Features Invariant to Affine Transformations." *Ph.D. Thesis*, 2002.

References

[Mikolajczyk 2004] K. Mikolajczyk and C. Schmid. "Scale and Affine Invariant Interest Point Detectors." *IJCV*, 2004.

[Mikolajczyk 2005] K. Mikolajczyk and C. Schmid. "A Performance Evaluation of Local Descriptors." *PAMI*, 2005.

[SIFT] SIFT Binaries. <http://www.cs.ubc.ca/~lowe/keypoints/>

[Witkin 1983] A. Witkin. "Scale-Space Filtering". *IJCAI*, 1983.